



РЭУ.РФ
РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ Г.В. ПЛЕХАНОВА

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего
образования
«**Российский экономический университет имени Г.В. Плеханова**»

Пермский институт (филиал)

Отчет по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____
(подпись)

_____Николаева Татьяна Сергеевна
(фамилия, имя, отчество)

Группы _____ИПо-42

Руководитель практической подготовки от профильной организации

_____Руководитель ТО
(должность)

(подпись)

_____Черняков Илья Юрьевич
(фамилия, имя, отчество)

МП

«16» марта 2026 года

Руководитель практической подготовки от института

(подпись)

_____Берестов Дмитрий Борисович
(фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

Содержание

ВВЕДЕНИЕ.....	3
РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ	5
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии	5
РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ.....	7
Тема 2.1. Ознакомление с организацией работы на предприятии или структурном подразделении	7
Тема 2.2. Ознакомление с должностными и функциональными обязанностями	11
РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ.....	16
Тема 3.1. Выполнение производственных заданий	16
3.1.1. Использование выбранной системы контроля версий	16
3.1.2. Разработка программного обеспечения	17
3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества;	43
3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;	44
3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств;.....	47
3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения;	61
3.1.7 Интегрирование программных модулей;	62
3.1.8. Верификация и аттестация программного обеспечения	65
ЗАКЛЮЧЕНИЕ	68
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	69

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;
5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
7. Планировать и реализовывать собственное профессиональное и личностное развитие;
8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами
9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
10. Использовать информационные технологии в профессиональной деятельности
11. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

Требования безопасности на производственных и офисных помещениях:

1. Знать расположение первичных средств пожаротушения (огнетушители) и путей эвакуации.
2. Соблюдать чистоту и порядок на рабочем месте, не загромождать проходы.
3. Запрещается проводить работы в состоянии алкогольного или наркотического опьянения.
4. О всех неисправностях оборудования, приборов и электропроводки немедленно сообщать руководителю практики.

При работе с ПК:

1. Следите за целостностью кабелей питания и периферии.
 2. Не размещайте жидкости в непосредственной близости от клавиатуры, системного блока и мониторов.
 3. Соблюдайте правила эргономики для сохранения осанки и зрения.
- Действия в аварийных ситуациях.

При обнаружении дыма, запаха гари или возгорания:

1. Немедленно отключите питание оборудования (если это безопасно).
2. Сообщите в пожарную охрану по телефону и руководителю.
3. Приступите к тушению возгорания с помощью огнетушителя (углекислотного или порошкового).

При поражении человека электрическим током:

1. Немедленно отключите источник питания (выдерните вилку из

розетки, выключите рубильник).

2. Вызовите скорую помощь и окажите первую помощь до ее приезда.

3. Немедленно сообщите руководителю.

Требования по окончании работы:

1. Отключить от сети все электрооборудование (ПК, лабораторные блоки питания).

2. Привести в порядок рабочее место.

3. Тщательно вымыть руки с мылом.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

Тема 2.1. Ознакомление с организацией работы на предприятии или структурном подразделении

Общая характеристика организации: специализируется на поставке, установке и сервисном обслуживании комплексных систем безопасности на основе современных достижений в этой области. Организация эффективно работает с 2006 г., имеет за плечами достойный опыт в решении сложных задач и широкую клиентскую базу

Роль информационных технологий в бизнес-процессах компании:

Для компании домофонных систем Информационные технологии — это не поддержка, а ядро бизнеса. Они позволяют:

- Автоматизировать рутинные процессы (продажи, обслуживание).
- Контролировать работу тысяч устройств удаленно.
- Масштабировать бизнес без пропорционального роста издержек.
- Создавать и монетизировать инновационные услуги, уходя от конкуренции только по цене железа.

Без глубокой интеграции ИТ такая компания сегодня не может быть ни эффективной, ни конкурентоспособной.

1. Режим работы

Режим работы предприятия определяет временные рамки его функционирования и основывается на производственной необходимости, а также соблюдении трудового законодательства.

Как правило, устанавливается пятидневная рабочая неделя с двумя выходными днями (суббота и воскресенье). Для производственных и операционных подразделений может действовать сменный график (2/2, 3/3) для обеспечения непрерывности процессов.

Продолжительность рабочего дня:

— 8 часов в день (40 часов в неделю). Конкретное время начала и окончания работы (например, с 9:00 до 18:00) фиксируется в Правилах внутреннего трудового распорядка.

Обеденный перерыв:

Предоставляется перерыв для отдыха и питания продолжительностью 1 час, который не включается в рабочее время. Гибкий график: Для некоторых категорий сотрудников (например, специалистов проектной работы) по согласованию с руководством может быть установлен гибкий график или удаленная работа.

2. Форма организации труда и Правила внутреннего распорядка

Это система правил, регламентирующих трудовые отношения и поведение сотрудников на предприятии.

Формы организации труда:

- Индивидуальная: Работник выполняет задачи самостоятельно на своем рабочем месте.
- Бригадная (командная): Группа сотрудников объединяется для выполнения общей задачи, неся коллективную ответственность за результат.
- Проектная: Создаются временные команды для реализации конкретных проектов. После завершения проекта команда расформировывается.

Правила внутреннего трудового распорядка (ПВТР) — это ключевой внутренний документ, который включает:

- Порядок приема, перевода и увольнения сотрудников.
- Основные права и обязанности работника и работодателя.
- Режим рабочего времени и времени отдыха.
- Меры поощрения за успехи в работе (благодарности, премии).
- Дисциплинарные взыскания за нарушения (замечание, выговор, увольнение).

- Ответственность за сохранность коммерческой тайны и имущества компании.

- Правила поведения на территории предприятия, требования к внешнему виду (дресс-код).

3. Структурные подразделения предприятия

Организационная структура — это схема, отражающая состав, иерархию и взаимосвязи между различными подразделениями компании. Линейные подразделения: Отвечают за основные производственные или операционные процессы.

Функциональные подразделения: Выполняют специализированные задачи по поддержке основной деятельности (например, *Отдел кадров, Бухгалтерия, Юридический отдел, IT-отдел*).

Матричные/проектные подразделения: Временные структуры, создаваемые для решения конкретных задач с привлечением специалистов из разных функциональных отделов.

Пример иерархии:

1. Генеральный директор
2. Заместители генерального директора (по производству, по финансам, по развитию)

3. Департаменты/Управления (Управление производства, Финансовый департамент, Коммерческий департамент)

4. Отделы/Службы внутри департаментов (Отдел логистики, Отдел маркетинга, Планово-экономический отдел)

5. Секторы/Группы внутри отделов.

4. Штатное расписание

Штатное расписание — это организационно-распорядительный документ, который закрепляет структуру и численность компании. Содержит перечень должностей, количество штатных единиц по каждой должности, размеры окладов (тарифных ставок), возможные надбавки и фонд оплаты труда (ФОТ) в целом по компании.

Назначение:

- Формализация структуры и численности персонала.
- Планирование расходов на оплату труда.
- Основа для приема сотрудников на конкретные должности.

Утверждается: Приказом руководителя предприятия и является конфиденциальным документом.

5. Принципы управления

Это фундаментальные правила и убеждения, на которых строится вся система управления компанией.

Принцип единоначалия: Каждый сотрудник подчиняется только одному непосредственному руководителю.

Принцип делегирования полномочий: Руководитель передает часть своих задач и ответственности подчиненным для повышения эффективности.

Принцип иерархии: Четкое распределение прав и ответственности по уровням управления.

Принцип обратной связи: Руководство регулярно получает информацию о результатах работы и оперативно реагирует на них.

Принцип мотивации и стимулирования: Создание системы, побуждающей сотрудников работать эффективно для достижения целей компании.

Принцип ответственности: Каждый сотрудник несет ответственность за выполнение своих задач в установленные сроки и с надлежащим качеством.

6. Руководство и осуществление должностных обязанностей

Это система, определяющая, как осуществляется управление и как сотрудники выполняют свою работу.

Руководство:

- Высшее руководство (Генеральный директор, Совет директоров) определяет стратегию и общее направление развития компании.

- Среднее звено (руководители департаментов, начальники отделов) организуют работу своих подразделений для достижения стратегических целей.

- Линейные руководители (мастера, старшие смен) управляют операционной деятельностью сотрудников на местах.

Осуществление должностных обязанностей:

- Регламентируется должностными инструкциями — документами, которые четко определяют для каждой должности: права, обязанности, ответственность, квалификационные требования и кому непосредственно подчиняется сотрудник.

- Выполнение задач осуществляется в соответствии с внутренними регламентами, стандартами и политиками компании.

- Контроль за исполнением обязанностей осуществляется непосредственным руководителем через регулярные отчеты, планерки и оценку ключевых показателей эффективности (KPI).

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

Основные обязанности сотрудника:

1. Добросовестно исполнять свои трудовые обязанности, возложенные на него трудовым договором и должностной инструкцией.

2. Соблюдать Правила внутреннего трудового распорядка, требования по охране труда и обеспечению безопасности труда.

3. Бережно относиться к имуществу работодателя (включая оборудование, программное обеспечение, базы данных) и других сотрудников.

4. Выполнять установленные нормы труда.

5. Незамедлительно сообщать непосредственному руководителю о возникновении ситуации, представляющей угрозу жизни и здоровью людей, сохранности имущества компании, в том числе о неисправности оборудования или программного обеспечения.

6. Сохранять коммерческую и конфиденциальную тайну компании, не разглашать внутреннюю информацию, включая данные о клиентах, партнерах, исходный код программ и алгоритмы работы систем.

7. Не допускать действий, которые могут нанести ущерб деловой репутации или коммерческим интересам компании.

Основные права сотрудника:

1. На заключение, изменение и расторжение трудового договора в порядке и на условиях, установленных Трудовым кодексом РФ.

2. На предоставление ему работы, обусловленной трудовым договором.

3. На рабочее место, соответствующее государственным нормативным требованиям охраны труда.

4. На своевременную и в полном объеме выплату заработной платы.

5. На отдых, обеспечиваемый установлением нормальной продолжительности рабочего времени, выходных и нерабочих праздничных дней, оплачиваемых ежегодных отпусков.

6. На полную достоверную информацию об условиях труда и требованиях охраны труда на рабочем месте.

7. На профессиональную подготовку, переподготовку и повышение своей квалификации в порядке, установленном Трудовым кодексом РФ.

8. На защиту своих трудовых прав, свобод и законных интересов всеми не запрещенными законом способами.

Общие положения

1.1. Настоящая должностная инструкция определяет функциональные обязанности, права и ответственность инженера-программиста микропрограммного обеспечения (далее – Программист).

1.2. Программист назначается на должность и освобождается от должности приказом Генерального директора по представлению Руководителя отдела разработки.

1.3. Программист подчиняется непосредственно Руководителю отдела разработки.

1.4. На должность Программиста назначается лицо, имеющее высшее образование (техническое/ИТ) и опыт работы в программировании встроенных систем (embedded systems) от 1 года.

1.5. Программист должен знать:

- Языки программирования C/C++ для встраиваемых систем.
- Архитектуру микроконтроллеров (семейства ARM Cortex-M, AVR, ESP32 и т.п.).
- Принципы работы периферийных интерфейсов: UART, I2C, SPI, USB, Ethernet.
- Протоколы передачи данных (в т.ч. для связи панели вызова с контроллером).
- Основы сетевых технологий (IP-адресация, TCP/UDP) для IP-домофонов.
- Принципы работы и кодирования аудио- и видеоданных.
- Средства отладки: JTAG, SWD, логические анализаторы, осциллографы.
- Системы контроля версий (Git).
- Английский язык на уровне чтения технической документации.

Должностные обязанности

Программист обязан:

2.1. Разрабатывать, тестировать и отлаживать микропрограммное обеспечение (прошивки) для домофонных панелей вызова, контроллеров шлюзов и другого оборудования.

2.2. Портить (адаптировать) существующее ПО под новые аппаратные платформы и модификации устройств.

2.3. Оптимизировать код для работы в условиях ограниченных ресурсов (память, тактовая частота).

2.4. Взаимодействовать с инженерами-схемотехниками и конструкторами для устранения аппаратных ошибок и совместной отладки устройств.

2.5. Разрабатывать и поддерживать техническую документацию: описание алгоритмов, API, инструкции по сборке ПО.

2.6. Участвовать в создании и поддержке инструментов для автоматизированного тестирования аппаратно-программных комплексов.

2.7. Проводить анализ и исправление ошибок (bug fixing), выявленных на этапах тестирования и эксплуатации продукции.

2.8. Обеспечивать соблюдение сроков выполнения задач в рамках установленного плана разработки.

Программист имеет право:

3.1. Запрашивать и получать от других подразделений компании информацию и документы, необходимые для выполнения своих должностных обязанностей.

3.2. Вносить предложения по совершенствованию работы, связанной с обязанностями, предусмотренными настоящей инструкцией, в том числе по улучшению программного кода, инструментов разработки и процессов.

3.3. Участвовать в совещаниях, на которых рассматриваются вопросы, связанные с его профессиональной деятельностью.

3.4. Требовать от руководства обеспечения организационно-технических условий, необходимых для исполнения должностных обязанностей (исправное оборудование, лицензионное ПО, доступ к стендам для тестирования).

Программист несет ответственность:

4.1. За ненадлежащее исполнение или неисполнение своих должностных обязанностей, предусмотренных настоящей инструкцией, — в пределах, определенных действующим трудовым законодательством РФ.

4.2. За правонарушения, совершенные в процессе своей деятельности, — в пределах, определенных действующим административным, уголовным и гражданским законодательством РФ.

4.3. За причинение материального ущерба — в пределах, определенных действующим трудовым и гражданским законодательством РФ.

4.4. За разглашение коммерческой тайны и конфиденциальной информации о проектах компании.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Выполнение производственных заданий

3.1.1. Использование выбранной системы контроля версий

Из всех систем был использован Git. Git – это распределённая система контроля версий, которая позволяет отслеживать изменения в коде или других файлах, управлять версиями проекта и координировать работу между несколькими разработчиками. Она была создана Линусом Торвальдсом в 2005 году для разработки ядра Linux и с тех пор стала одной из самых популярных систем контроля версий благодаря своей гибкости, скорости и надёжности.

Как работает Git:

1. Локальный репозиторий: Git хранит всю историю изменений проекта в локальном репозитории на компьютере пользователя. Это позволяет работать с проектом даже без подключения к интернету;

2. Коммиты: Изменения в проекте фиксируются с помощью коммитов – «снимков» состояния файлов на определённый момент. Каждый коммит имеет уникальный идентификатор (хэш), описание и ссылку на предыдущий коммит, что создаёт цепочку версий;

3. Ветки (branches): Git позволяет создавать ветки для параллельной разработки. Например, можно работать над новой функцией в отдельной ветке, не затрагивая основную версию проекта (обычно ветка main или master). После завершения работы ветки можно объединить (merge).

4. Удалённый репозиторий: для совместной работы Git поддерживает синхронизацию с удалёнными репозиториями (например, на GitHub, GitLab или Bitbucket). Команды push и pull используются для отправки изменений на сервер и получения обновлений от других участников;

5. Индексация (staging area): перед фиксацией изменений Git требует добавить их в промежуточную область (индекс) с помощью команды `git add`. Это позволяет выбирать, какие именно изменения войдут в следующий коммит;

6. Конфликты и их разрешение: если несколько человек изменяют один и тот же участок кода, могут возникнуть конфликты. Git помогает выявить такие ситуации и предоставляет инструменты для их ручного разрешения.

Основные команды Git:

- `git init` – инициализирует новый репозиторий;
- `git clone` – копирует существующий репозиторий;
- `git add` – добавляет изменения в индекс;
- `git commit` – фиксирует изменения в локальном репозитории;
- `git push` – отправляет изменения в удалённый репозиторий;
- `git pull` – загружает изменения из удалённого репозитория;
- `git branch` – управляет ветками;
- `git merge` – объединяет ветки.

Преимущества Git:

- высокая скорость работы благодаря локальному хранению данных;
- поддержка нелинейной разработки через ветвление;
- надёжность: данные сложно потерять из-за распределённой природы системы;
- гибкость в управлении историей изменений (например, через `rebase` или `cherry-pick`).

3.1.2. Разработка программного обеспечения

3.1.2.1. Выбор модели разработки

Для разработки программного обеспечения была выбрана **спиральная модель**.

Обоснование выбора модели разработки:

Спиральная модель была выбрана благодаря ее гибкости и акценту на управлении рисками, что особенно важно для проектов средней и высокой

сложности. Этот подход позволяет на каждом этапе (или "витке" спирали) проводить анализ требований, проектирование, разработку и тестирование, а также оценивать риски и корректировать план действий. Такой метод идеально подходит для проектов, где требования могут уточняться в процессе работы, а также для обеспечения высокого качества конечного продукта за счет постепенного улучшения и проверки на каждом цикле. В отличие от каскадной модели, спиральная модель допускает итеративное развитие, что снижает вероятность крупных ошибок на поздних стадиях. По сравнению с V-образной моделью, она более гибкая, так как не требует строгой последовательности этапов и позволяет адаптироваться к изменениям. Модель прототипирования и RAD были отклонены из-за их ориентации на быстрые результаты с меньшим вниманием к долгосрочным рискам, а итерационная модель уступила спиральной из-за недостаточного акцента на анализе рисков.

3.1.2.2. Инструменты, использованные для создания продукта

1. Visual Studio

- основной инструмент для написания, отладки и тестирования кода;
- использовались встроенные возможности, такие как IntelliSense для автодополнения кода, отладчик для поиска ошибок и менеджер проектов для управления структурой решения.

2. Git

- система контроля версий для отслеживания изменений в коде и совместной работы (если применимо).

3. NuGet Package Manager

- использовался для подключения дополнительных библиотек и зависимостей, необходимых для работы с .NET 9.0.

4. MSBuild

- инструмент сборки, встроенный в Visual Studio, для компиляции и развертывания проекта.

5. Тестировочные утилиты (например, NUnit или встроенный MSTest)

– применялись для написания и выполнения модульных тестов на каждом витке спирали, что соответствует выбранной модели разработки.

3.1.2.3 Среда программирования

Разработка велась в среде **Microsoft Visual Studio** с использованием языка программирования **C#** на платформе **.NET Framework 4.8**.

– **Visual Studio:** Интегрированная среда разработки (IDE), предоставляющая мощный набор инструментов для написания, отладки и тестирования кода. Она включает редактор кода с поддержкой подсветки синтаксиса, автодополнения и рефакторинга, а также встроенные средства для работы с проектами .NET. Выбор Visual Studio обусловлен ее полной совместимостью с C# и .NET Framework, а также широкими возможностями для управления проектами любой сложности.

– **C#:** Объектно-ориентированный язык программирования.

– **.NET 9.0:** последняя версия классического .NET Framework, обеспечивающая стабильность, обширный набор готовых классов и совместимость с большинством современных библиотек.

3.1.2.4. Этапы разработки программного обеспечения

Выбор шаблона

На этом этапе были выбраны подходящие шаблоны проектов в Visual Studio для создания компонентов системы.

Процесс:

– Для работы с базой данных и бизнес-логикой выбран шаблон «Class Library (.NET Framework)» – создан проект NikolaevaLib.

– Для тестирования методов библиотеки выбран шаблон «Unit Test Project (.NET Framework)» – создан проект NikolaevaTests.

– Для пользовательского интерфейса выбран шаблон «WPF App (.NET Framework)» – создан проект NikolaevaWPF.

- Все проекты настроены под .NET 9.0 для обеспечения совместимости и использования возможностей платформы.
- Подключены необходимые зависимости через NuGet: Npgsql для работы с PostgreSQL в NikolaevaLib.

Составление схемы информационной базы

На этом этапе была спроектирована и реализована база данных nikolaeva_db в PostgreSQL, созданы необходимые таблицы в схеме app, а также подготовлены программные модели для работы с данными в проекте NikolaevaLib.

Процесс:

Настройка базы данных:

- Создана роль app с логином и паролем для управления доступом:

```
CREATE ROLE app WITH LOGIN PASSWORD '123456789';
```

- Создан табличный пространство nikolaeva_tb с владельцем app и указанием расположения данных:

```
CREATE TABLESPACE nikolaeva_tb OWNER app LOCATION 'C:\PR';
```

- Создана база данных nikolaeva_db с владельцем app и привязкой к табличному пространству:

```
CREATE DATABASE nikolaeva_db OWNER app TABLESPACE  
nikolaeva_tb;
```

- Подключение к базе данных выполнено от имени роли app:

```
\c nikolaeva_db app
```

- Создана схема app с авторизацией для роли app:

```
CREATE SCHEMA app AUTHORIZATION app;
```

Создание таблиц:

- **Таблица app.partner:**

Хранит информацию о партнерах с уникальным идентификатором id (автоинкремент), обязательными текстовыми полями (type, name, director, email, phone, legaladdress) и необязательным полем rating (целочисленное).

– **Таблица app.discount:**

Содержит данные о скидках с уникальным id, ссылкой partnerid на таблицу partner (внешний ключ с опцией ON DELETE SET NULL) и полем percentage (число с плавающей точкой для процента скидки).

– **Таблица app.sale:**

Хранит информацию о продажах с уникальным id, ссылкой partnerid на таблицу partner (внешний ключ с опцией ON DELETE SET NULL), обязательными полями quantity (количество, целое число), date (дата) и product_name (название продукта, строка).

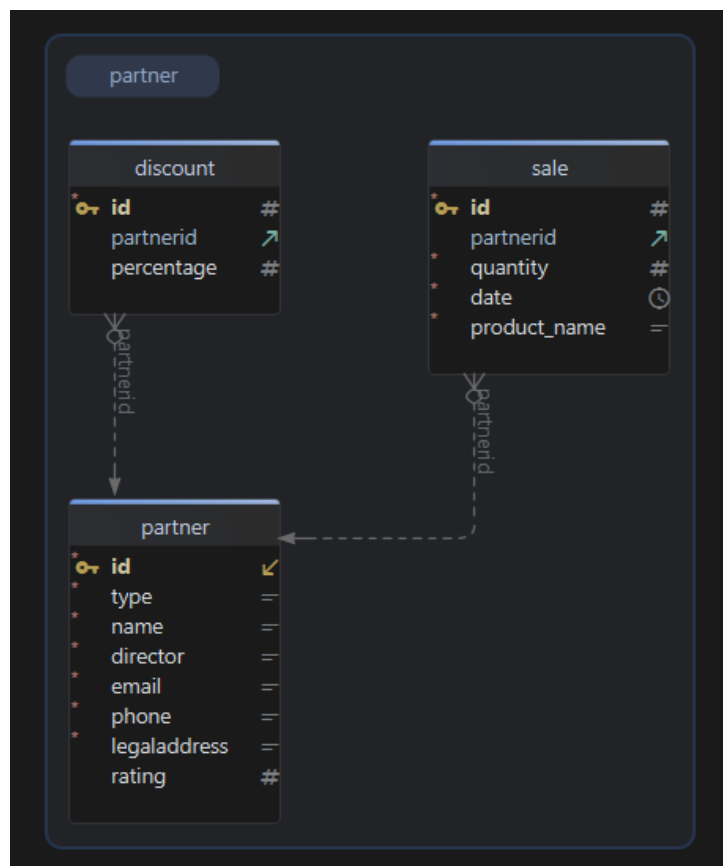


Рисунок 1 – Ег-Диаграмма

Создание библиотеки классов

Подготовка окружения

– Я начала с того, что создала проект в Visual Studio, выбрав шаблон библиотеки классов (.NET). Для работы с Entity Framework Core и

PostgreSQL мне нужно было установить нужные пакеты. Я решила сделать это через интерфейс Visual Studio, а не командную строку.

Открыла Tools → NuGet Package Manager → Manage NuGet Packages for Solution и в поиске нашла:

- Microsoft.EntityFrameworkCore – для работы с EF Core.
- Npgsql.EntityFrameworkCore.PostgreSQL – драйвер для PostgreSQL.

Выбрала каждый пакет, нажала **Install**, и Visual Studio добавила их в проект.

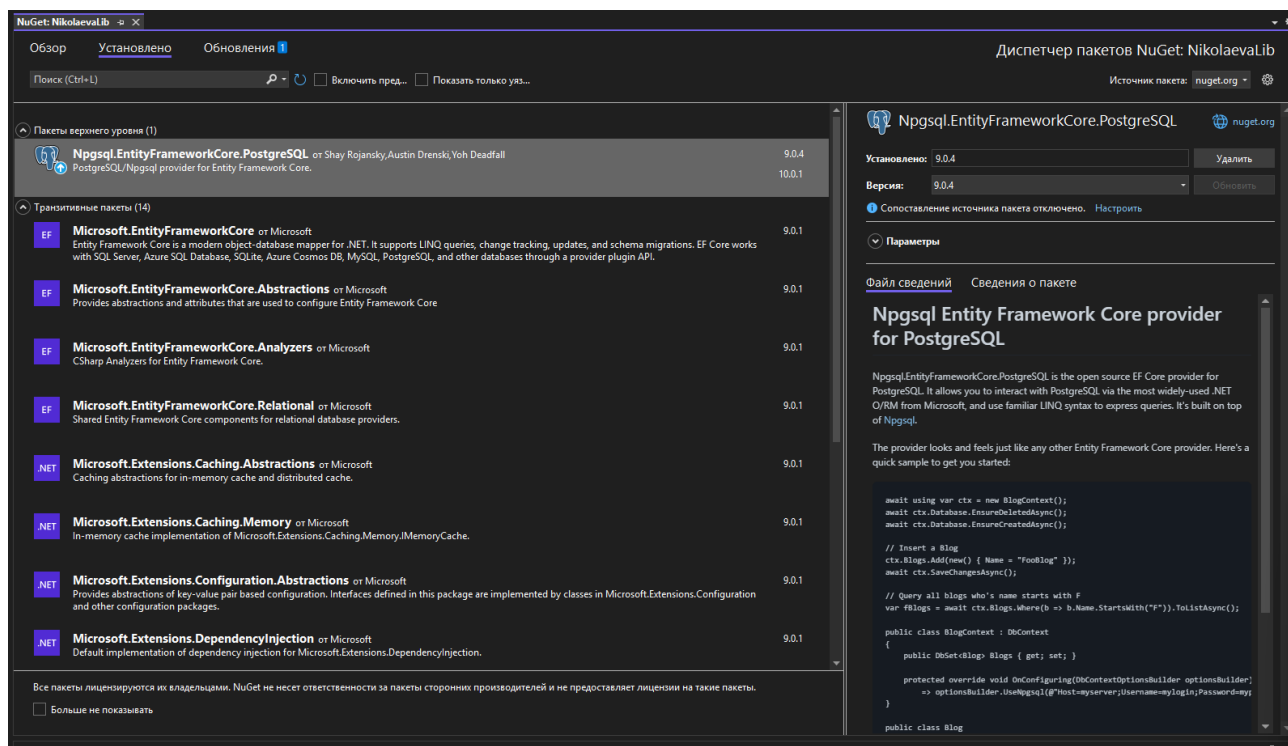


Рисунок 2 – NuGet Package Manager

Запуск Scaffold-DbContext

После установки пакетов я открыла Package Manager Console через Tools → NuGet Package Manager → Package Manager Console и выполнила команду: Scaffold-DbContext

"Host=localhost;Database=nikolaeva_db;Username=app;Password=123456789

"

Npgsql.EntityFrameworkCore.PostgreSQL -Context ApplicationContext

– Строка подключения:

"Host=localhost; Database=nikolaeva_db; Username=app; Password=123456789" – данные для подключения к моей локальной базе PostgreSQL.

- **Провайдер:** Npgsql.EntityFrameworkCore.PostgreSQL – указала, что использую PostgreSQL.

- **-Context ApplicationDbContext:** Назвала контекст ApplicationDbContext.

Что получилось после генерации:

После выполнения команды в корне проекта появились файлы: ApplicationDbContext.cs, Discount.cs, Partner.cs и Sale.cs. Visual Studio сгенерировала их на основе структуры базы nikolaeva_db.

Например:

- В ApplicationDbContext.cs появился метод OnConfiguring с моей строкой подключения:

```
optionsBuilder.UseNpgsql("Host=localhost;Database=nikolaeva_db;Username=app;Password=123456789");
```

- В OnModelCreating были настроены таблицы (discount, partner, sale) со схемой app, а также связи между ними и внешние ключи.

Ручная доработка:

После генерации я добавила свои классы:

- PartnerView.cs – для удобного представления данных о партнёрах.
- Service.cs – написала там методы вроде UpdateDiscounts и LoadPartners, чтобы реализовать нужную логику.

С кодом можно ознакомиться в проекте.

Полученная диаграмма классов (см. Рис. 3):

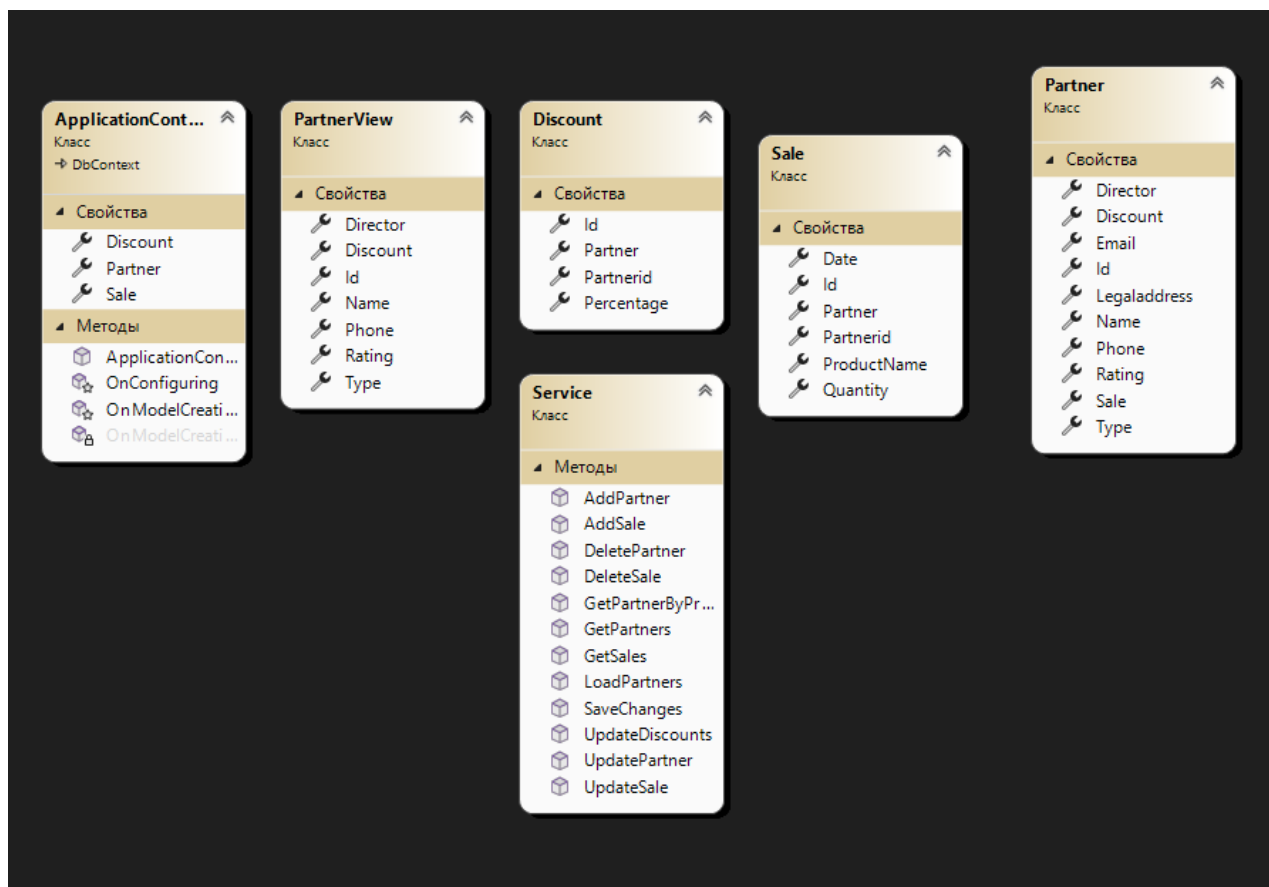


Рисунок 3 – Диаграмма классов

Тестирование

Я создала новый проект в Visual Studio, выбрав шаблон **Unit Test Project (.NET Framework)**, и назвала его NikolaevaTests. В этом проекте мне нужно было подключиться к моей библиотеке NikolaevaLib, так что я добавила ссылку на неё через **Project → Add Reference →** выбрала NikolaevaLib.

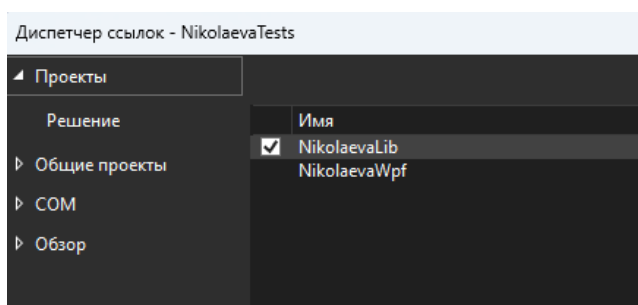


Рисунок 4 – Диспетчер ссылок

Настройка окружения для тестов

Я решила, что каждый тест будет работать с чистой базой данных, поэтому добавил методы инициализации и очистки:

– В [TestInitialize] я создавала экземпляры ApplicationContext и Service перед каждым тестом:

```
_db = new ApplicationContext();
```

```
_svc = new Service();
```

– В [TestCleanup] я удаляла все временные данные, которые начинались с префикса TMP_. Я искала партнёров с таким именем, удаляла связанные с ними скидки и продажи, а потом самих партнёров

Написание тестов

Я написала несколько тестов, чтобы проверить основные методы из Service. Вот как я это делал:

– Тест добавления партнёра (AddPartner)

Я создала нового партнёра с префиксом TMP_, добавил его через

_svc.AddPartner, а потом проверил, появился ли он в базе:

```
public void AddPartner()
{
    var p = new Partner { Name = Prefix + "Alpha", Type = "Retail", Director =
"Alice", Email = "alpha@example.com", Phone = "111222333", Legaladdress = "City A",
Rating = 4 };
    _svc.AddPartner(_db, p);

    var res = _db.Partner.FirstOrDefault(x => x.Name == p.Name);
    Assert.IsNotNull(res);
    Assert.AreEqual("Retail", res.Type);
}
```

– Тест обновления партнёра (UpdatePartner)

Я добавила партнёра, изменил его имя и email, обновила через _svc.UpdatePartner и проверила, что данные в базе обновились:

```
public void UpdatePartner()
```

```
{
```

```

        var p = new Partner { Name = Prefix + "Old", Type = "Wholesale", Director =
"Bob", Email = "old@example.com", Phone = "444555666", Legaladdress = "City B",
Rating = 3 };
        _db.Partner.Add(p);
        _db.SaveChanges();

        p.Name = Prefix + "New";
        p.Email = "new@example.com";
        _svc.UpdatePartner(_db, p);

        var res = _db.Partner.FirstOrDefault(x => x.Id == p.Id);
        Assert.IsNotNull(res);
        Assert.AreEqual(Prefix + "New", res.Name);
        Assert.AreEqual("new@example.com", res.Email);
    }

```

– Тест удаления партнёра (DeletePartner)

Я добавила партнёра, удалила его через `_svc.DeletePartner` и убедилась, что он исчез из базы:

```

public void DeletePartner()
{
    var p = new Partner { Name = Prefix + "ToRemove", Type = "Retail", Director =
"Charlie", Email = "remove@example.com", Phone = "777888999", Legaladdress = "City
C", Rating = 5 };
    _db.Partner.Add(p);
    _db.SaveChanges();

    _svc.DeletePartner(_db, p);
    var res = _db.Partner.FirstOrDefault(x => x.Name == p.Name);
    Assert.IsNull(res);
}

```

– Тест обновления скидок (UpdateDiscounts)

Я добавила партнёра и продажу с количеством 50 000, вызвала `_svc.UpdateDiscounts` и проверила, что скидка стала 5%:

```

public void UpdateDiscounts()
{
    var p = new Partner { Name = Prefix + "Beta", Type = "Retail", Director =
"Dave", Email = "beta@example.com", Phone = "123321123", Legaladdress = "City D",
Rating = 2 };
    _db.Partner.Add(p);
    _db.SaveChanges();
}

```

```

        _db.Sale.Add(new Sale { Partnerid = p.Id, Quantity = 50000, Date =
DateTime.Now, ProductName = "Item X" });
        _db.SaveChanges();

        _svc.UpdateDiscounts(_db);
        var d = _db.Discount.FirstOrDefault(x => x.Partnerid == p.Id);
        Assert.IsNotNull(d);
        Assert.AreEqual(5, d.Percentage);
    }

```

– Тест поиска партнёра по имени (GetPartner)

Я добавила партнёра и нашла его через `_svc.GetPartnerByProperties`, проверив, что вернулся правильный объект:

```

public void GetPartner()
{
    var p = new Partner { Name = Prefix + "Gamma", Type = "Wholesale", Director =
"Eve", Email = "gamma@example.com", Phone = "987654321", Legaladdress = "City E",
Rating = 1 };
    _db.Partner.Add(p);
    _db.SaveChanges();

    var res = _svc.GetPartnerByProperties(_db, p.Name);
    Assert.IsNotNull(res);
    Assert.AreEqual(p.Name, res.Name);
}

```

– Тест загрузки списка партнёров (LoadPartners)

Я добавила партнёра с продажей, обновила скидки и проверила, что он попал в список через `_svc.LoadPartners`:

```

public void LoadPartners()
{
    var p = new Partner { Name = Prefix + "Delta", Type = "Retail", Director =
"Frank", Email = "delta@example.com", Phone = "741852963", Legaladdress = "City F",
Rating = 3 };
    _db.Partner.Add(p);
    _db.SaveChanges();

    _db.Sale.Add(new Sale { Partnerid = p.Id, Quantity = 45000, Date =
DateTime.Now, ProductName = "Item Y" });
    _db.SaveChanges();

    _svc.UpdateDiscounts(_db);
    var res = _svc.LoadPartners(_db);
}

```

```
Assert.IsTrue(res.Any(x => x.Name == p.Name));
}
```

– Тест сохранения данных SaveChanges

Добавила партнера и попробовала его найти:

```
public void SaveChanges()
{
    var p = new Partner { Name = Prefix + "Sigma", Type = "Retail", Director =
"George", Email = "sigma@example.com", Phone = "555666777", Legaladdress = "City G",
Rating = 5 };
    _db.Partner.Add(p);
    _svc.SaveChanges(_db);
    var res = _db.Partner.FirstOrDefault(x => x.Name == p.Name);
    Assert.IsNotNull(res);
    Assert.AreEqual("Retail", res.Type);
}
```

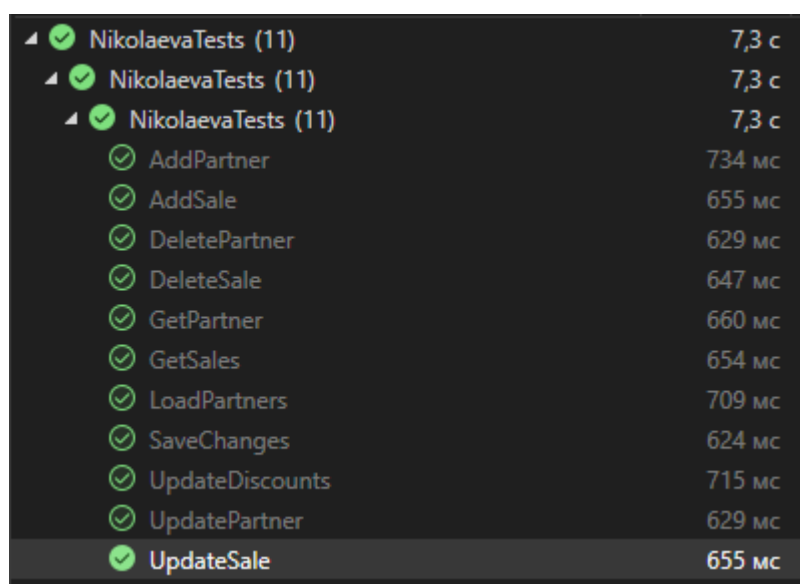
– Остальные тесты

Другие тесты содержат схожую логику, что я описывала ранее. Со всеми тестами можно ознакомиться в проекте.

Запуск тестов

После написания тестов я открыла **Test Explorer** в Visual Studio (через **Test → Test**

Explorer) и провела каждый тест по отдельности т.к одновременно методы будут работать некорректно. Все тесты прошли успешно, что показало, что методы в Service работают как надо (см. Рис. 5).



▲ ✓ NikolaevaTests (11)	7,3 с
▲ ✓ NikolaevaTests (11)	7,3 с
▲ ✓ NikolaevaTests (11)	7,3 с
✓ AddPartner	734 мс
✓ AddSale	655 мс
✓ DeletePartner	629 мс
✓ DeleteSale	647 мс
✓ GetPartner	660 мс
✓ GetSales	654 мс
✓ LoadPartners	709 мс
✓ SaveChanges	624 мс
✓ UpdateDiscounts	715 мс
✓ UpdatePartner	629 мс
✓ UpdateSale	655 мс

Рисунок 5 – Тесты

Разработка интерфейса WPF и его функционала.

В ходе разработки были выполнены следующие действия:

– Выбор шаблона:

В меню выбрал **File** → **New** → **Project**, в поиске ввёл "WPF" и выбрал шаблон **WPF App (.NET Framework)**. Нажал **Next**.

Настройка проекта:

Назвала проект NikolaevaWPF, в качестве фреймворка. Нажала **Create**.

Добавление зависимостей:

Моя библиотека NikolaevaLib уже была готова, так что я добавила её в проект. Щёлкнул правой кнопкой по NikolaevaWPF в Solution Explorer → **Add** → **Reference** → в разделе **Projects** выбрал NikolaevaLib → **OK**.

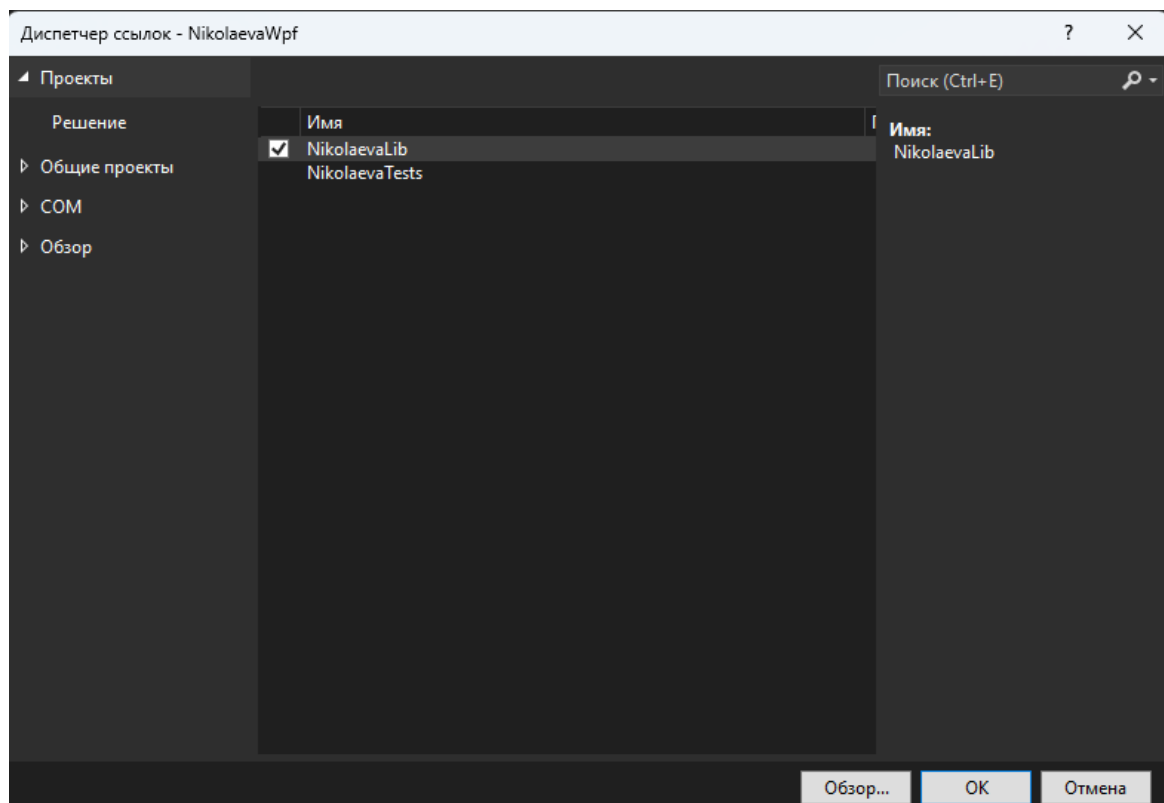


Рисунок 6 – Тесты

Главное окно (MainWindow):

Проектирование интерфейса Я продумала, каким должно быть окно:

– Цели:

Показывать партнёров и их продажи, давать возможность добавлять, редактировать и удалять данные.

– **Структура:**

Решила сделать меню сверху, список партнёров слева (примерно 300 пикселей шириной), таблицу продаж справа.

– **Функциональность:**

Меню для управления, список с выбором партнёра, таблица для отображения продаж, контекстные меню для удобства.

Прототипирование интерфейса

Я открыла MainWindow.xaml и начал с черновой версии:

– **Базовая разметка:**

Добавила <Grid> и <DockPanel>:

– **Меню:**

Создала простое меню:

```
<Menu DockPanel.Dock="Top">
  <MenuItem Header="Партнеры">
    <MenuItem Header="Добавить партнера"/>
  </MenuItem>
  <MenuItem Header="Продукция"/>
  <MenuItem Header="Выход"/> </Menu>
```

– **Список партнёров:**

Добавила <ListBox>:

```
<ListBox x:Name="PartnersList" Width="300">
  <ListBox.ItemTemplate>
    <DataTemplate>
      <TextBlock Text="{Binding Name}"/>
    </DataTemplate>
  </ListBox.ItemTemplate> </ListBox>
```

– **Таблица продаж:**

Поставила <DataGrid>:

```
<DataGrid x:Name="SalesDataGrid" AutoGenerateColumns="False">
  <DataGrid.Columns>
```

```

        <DataGridTextColumn Header="Наименование" Binding="{Binding
ProductName}"/>
        <DataGridTextColumn Header="Дата" Binding="{Binding Date}"/>
        <DataGridTextColumn Header="Кол-во" Binding="{Binding
Quantity}"/>
    </DataGrid.Columns>
</DataGrid>

```

Стилизация интерфейса

После прототипа я улучшила внешний вид:

– Окно:

```

Задал      Height="600"      Width="800"
ResizeMode="NoResize"
WindowStartupLocation="CenterScreen".

```

– Меню:

Добавила MinHeight="25", больше пунктов и разделители:

```

<Menu DockPanel.Dock="Top" MinHeight="25">
    <MenuItem Header="Партнеры">
        <MenuItem Header="Добавить партнера" Click="AddPartner_Click"/>
<Separator/>
        <MenuItem Header="Удалить партнера" Click="DeletePartner_Click"/>
    </MenuItem> </Menu>

```

– Список партнёров:

Улучшила шаблон с рамкой и двумя колонками:

```

<Border      BorderBrush="Gray"      BorderThickness="1"      Margin="5"
Padding="10">
    <Grid Width="240">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*"/>
            <ColumnDefinition Width="Auto"/>

```

```

</Grid.ColumnDefinitions>
<StackPanel Grid.Column="0">
    <TextBlock FontWeight="Bold">
        <Run Text="{Binding Type}"/> <Run Text=" | "/> <Run
Text="{Binding Name}"/>
    </TextBlock>
</StackPanel>
<TextBlock Grid.Column="1" Text="{Binding Discount,
StringFormat={}{0}%}"
FontWeight="Bold"/>
</Grid> </Border>

```

Таблица продаж:

Настроила центрирование и стили:

```

<DataGrid x:Name="SalesDataGrid" IsReadOnly="True"
CanUserSortColumns="False">
    <DataGrid.Resources>
        <Style TargetType="TextBlock">
            <Setter Property="TextAlignment" Value="Center"/>
        </Style>
    </DataGrid.Resources>
</DataGrid>

```

Добавил <TextBlock Text="Данные"> и <DataGrid.ContextMenu>.

В итоге получился такой вот интерфейс (см. Рис. 7):

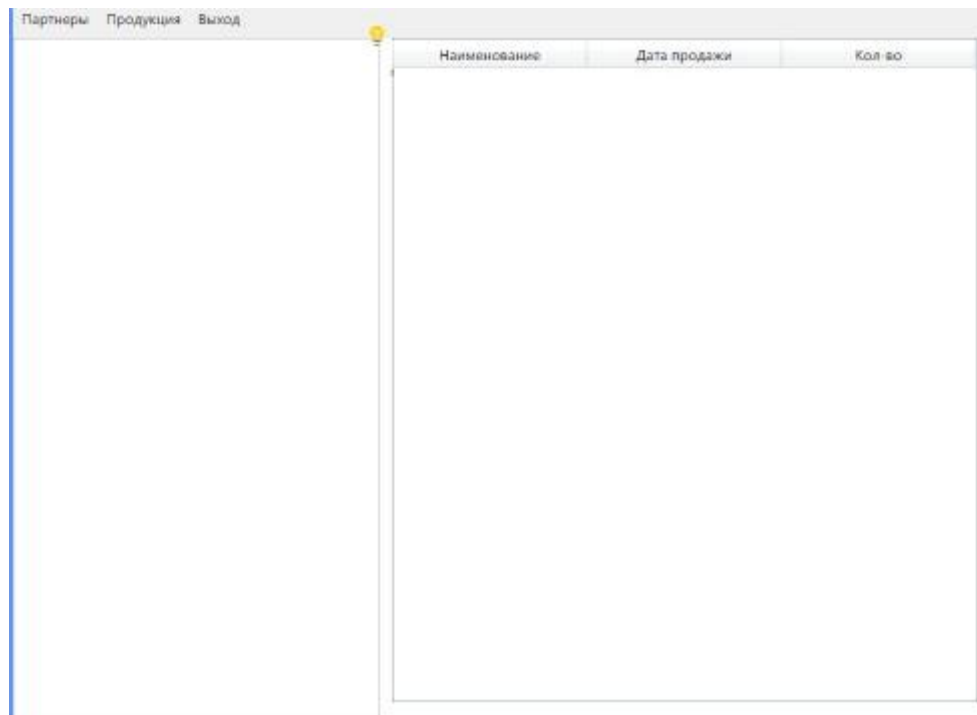


Рисунок 7 – Главное окно

Реализация методов в MainWindow.xaml.cs Я перешла к MainWindow.xaml.cs и написала код:

```
private readonly ApplicationContext _dbContext; // Контекст базы данных
private Service _partnerManager; // Сервис для управления партнёрами
Ссылка 0
public MainWindow()
{
    InitializeComponent();
    _dbContext = new ApplicationContext(); // Инициализация контекста базы данных
    _partnerManager = new Service(); // Инициализация сервиса для работы с партнёрами
    PartnersList.SelectionChanged += (s, e) => RefreshSalesView(); // Обработчик изменения выбора партнёра
    RefreshPartnersView(); // Обновление списка партнёров
    RefreshSalesView(); // Обновление списка продаж
}
```

Рисунок 8 – Инициализация

```
// Метод для обновления отображения партнёров
Ссылка 7
private void RefreshPartnersView()
{
    var currentSelectedPartner = PartnersList.SelectedItem as PartnerView; // Получаем текущего выбранного партнёра
    _partnerManager.UpdateDiscounts(_dbContext); // Обновляем скидки
    var partnerCollection = _partnerManager.LoadPartners(_dbContext); // Загружаем список партнёров
    PartnersList.ItemsSource = partnerCollection; // Обновляем источник данных для списка партнёров
    if (currentSelectedPartner != null)
    {
        PartnersList.SelectedItem = partnerCollection.FirstOrDefault(p => p.Name == currentSelectedPartner.Name); // Восстанавливаем выбор партнёра, если он был
    }
}
```

Рисунок 9 – Обновление партнеров

```

// Метод для обновления отображения продаж
Ссылка: 5
private void RefreshSalesView()
{
    var currentSelectedPartner = PartnersList.SelectedItem as PartnerView; // Получаем текущего выбранного партнёра
    if (currentSelectedPartner != null)
    {
        var partnerData = _partnerManager.GetPartnerByProperties(_databaseContext, currentSelectedPartner.Name); // Загружаем данные партнёра
        if (partnerData != null)
        {
            SalesDataGrid.ItemsSource = _partnerManager.GetSales(_databaseContext, partnerData); // Загружаем и отображаем продажи
        }
    }
    else
    {
        SalesDataGrid.ItemsSource = null; // Если партнёр не выбран, очищаем список продаж
    }
}

```

Рисунок 10 – Обновление продаж

```

// Метод для удаления партнёра
Ссылка: 2
private void DeletePartner_Click(object sender, RoutedEventArgs e)
{
    var currentSelectedPartner = PartnersList.SelectedItem as PartnerView; // Получаем текущего выбранного партнёра
    Window parentWindow = Window.GetWindow(this); // Получаем родительское окно
    if (currentSelectedPartner != null)
    {
        var partnerData = _partnerManager.GetPartnerByProperties(_databaseContext, currentSelectedPartner.Name); // Загружаем данные партнёра
        var confirmationResult = MessageBox.Show(parentWindow, "Вы действительно хотите удалить выбранного партнера?", "Подтверждение",
            MessageBoxButton.YesNo, MessageBoxImage.Question); // Подтверждение удаления
        if (confirmationResult == MessageBoxResult.Yes)
        {
            _partnerManager.DeletePartner(_databaseContext, partnerData); // Удаляем партнёра
        }
    }
    RefreshPartnersView(); // Обновляем список партнёров
}

```

Рисунок 11 – Удаление партнера

– Методы для продаж:

Аналогично написала AddSale_Click, EditSale_Click, DeleteSale_Click, ВЫЗЫВАЯ

SaleWindow и обновляя оба представления.

Окно PartnerWindow

Проектирование интерфейса

Я задумала окно для добавления и редактирования партнёров:

– **Цели:** Ввод данных партнёра (тип, название, директор и т.д.) с проверкой корректности.

– **Структура:** Поля ввода вертикально, кнопки «Сохранить» и «Отменить» внизу справа.

– **Функциональность:** Поддержка как создания нового партнёра, так и редактирования существующего.

Прототипирование интерфейса

В PartnerWindow.xaml я начала с базовой разметки:

- Добавил <Grid> с <StackPanel> для вертикального размещения:

<Grid>

<StackPanel Margin="10">

<TextBlock Text="Тип партнера"/>

<ComboBox x:Name="TypeComboBox"/>

<TextBlock Text="Название"/>

<TextBox x:Name="NameTextBox"/>

</StackPanel> </Grid>

- Постепенно добавила остальные поля и кнопки: <TextBox> для каждого текстового поля и <Button> для действий.

- Запустила проект, вызвав окно из MainWindow через new PartnerWindow(null).Show(), чтобы проверить разметку.

Стилизация интерфейса

Я доработала внешний вид в PartnerWindow.xaml:

- **Окно:**

Задала Height="435" Width="400"

ResizeMode="NoResize"

WindowStartupLocation="CenterOwner", чтобы окно было компактным и открывалось по центру родительского.

Поля:

Добавила отступы (Margin="0,0,0,5" для <TextBlock>, Margin="0,0,0,10" для <TextBox> и <ComboBox>), чтобы всё выглядело аккуратно:

<TextBlock Text="Тип партнера" Margin="0,0,0,5"/>

<ComboBox x:Name="TypeComboBox" Margin="0,0,0,10">

<ComboBoxItem Content="ЗАО"/>

<ComboBoxItem Content="ПАО"/>

<ComboBoxItem Content="ООО"/>

<ComboBoxItem Content="ОАО"/> </ComboBox>

– **Кнопки:**

Поместила их в `<StackPanel Orientation="Horizontal" HorizontalAlignment="Right">` с отступами:

`<Button Content="Сохранить" Click="SaveButton_Click" Margin="0,0,10,0"/>`

`<Button Content="Отменить" Click="CancelButton_Click"/>`

В итоге получился такой интерфейс:

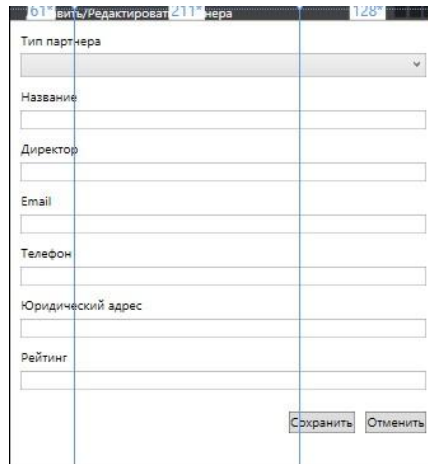


Рисунок 12 – Окно PartnerWindow

Реализация методов в PartnerWindow.xaml.cs

Я написала логику:

```
public partial class PartnerWindow : Window
{
    private readonly ApplicationContext _dbContext; // Контекст базы данных
    private readonly Service _partnerManager; // Сервис для работы с партнёрами
    private Partner _currentPartner; // Текущий редактируемый партнёр

    // Конструктор для добавления или редактирования партнёра
    Ссылка: 2
    public PartnerWindow(ApplicationContext context, Partner partnerToEdit = null)
    {
        InitializeComponent();
        _dbContext = context;
        _partnerManager = new Service();
        _currentPartner = partnerToEdit;

        // Если партнёр существует (редактирование), заполняем поля
        if (_currentPartner != null)
        {
            Title = "Редактирование партнера"; // Заголовок окна
            TypeComboBox.Text = _currentPartner.Type;
            NameTextBox.Text = _currentPartner.Name;
            DirectorTextBox.Text = _currentPartner.Director;
            EmailTextBox.Text = _currentPartner.Email;
            PhoneTextBox.Text = _currentPartner.Phone;
            LegalAddressTextBox.Text = _currentPartner.Legaladdress;
            RatingTextBox.Text = _currentPartner.Rating.ToString();
        }
        else
        {
            Title = "Добавление партнера"; // Заголовок окна для добавления
        }
    }
}
```

Рисунок 13 – Инициализация

– **Сохранение:**

Программный код:

```
private void SaveButton_Click(object sender, RoutedEventArgs e)
{
    try
    {
        // Проверка на пустые поля
        if (string.IsNullOrEmpty(NameTextBox.Text))
        {
            MessageBox.Show("Поле 'Название' не может быть пустым.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }
        if (string.IsNullOrEmpty(TypeComboBox.Text))
        {
            MessageBox.Show("Поле 'Тип' не может быть пустым.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }
        if (string.IsNullOrEmpty(DirectorTextBox.Text))
        {
            MessageBox.Show("Поле 'Директор' не может быть пустым.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }
        if (string.IsNullOrEmpty(EmailTextBox.Text) || !IsValidEmail(EmailTextBox.Text))
        {
            MessageBox.Show("Введите корректный адрес электронной почты (пример: example@mail.com).", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }
        if (string.IsNullOrEmpty(LegalAddressTextBox.Text))
        {
            MessageBox.Show("Поле 'Юридический адрес' не может быть пустым", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }
        if (string.IsNullOrEmpty(PhoneTextBox.Text) || !IsValidPhone(PhoneTextBox.Text))
        {
            return;
        }
    }
}
```

```

        MessageBox.Show("Введите номер телефона в формате +7XXXXXXXXXX
(пример: +79504693322).", "Ошибка ввода", MessageBoxButtons.OK,
MessageBoxImage.Error);
        return;
    }
    if (!int.TryParse(RatingTextBox.Text, out int partnerRating) ||
partnerRating <= 0)
    {
        MessageBox.Show("Рейтинг должен быть положительным числом.", "Ошибка
ввода", MessageBoxButtons.OK, MessageBoxImage.Error);
        return;
    }

    // Получаем выбранный тип партнёра
    var partnerType = (TypeComboBox.SelectedItem as
ComboBoxItem)?.Content.ToString();

    // Если партнёр не был передан (это новый партнёр), создаём новый объект
    if (_currentPartner == null)
    {
        var newPartnerRecord = new Partner
        {
            Type = partnerType,
            Name = NameTextBox.Text,
            Director = DirectorTextBox.Text,
            Email = EmailTextBox.Text,
            Phone = PhoneTextBox.Text,
            Legaladdress = LegalAddressTextBox.Text,
            Rating = partnerRating
        };

        _partnerManager.AddPartner(_databaseContext, newPartnerRecord); //
Добавляем нового партнёра
    }
    else
    {
        // Если партнёр редактируется, обновляем его данные
        _currentPartner.Type = partnerType;
        _currentPartner.Name = NameTextBox.Text;
        _currentPartner.Director = DirectorTextBox.Text;
        _currentPartner.Email = EmailTextBox.Text;
        _currentPartner.Phone = PhoneTextBox.Text;
        _currentPartner.Legaladdress = LegalAddressTextBox.Text;
        _currentPartner.Rating = partnerRating;
    }
}

```

```

        _partnerManager.UpdatePartner(_dbContext, _currentPartner); //
Обновляем партнёра в базе
    }

    this.DialogResult = true; // Закрываем окно с результатом
}
catch (Exception error)
{
    MessageBox.Show("Произошла ошибка: " + error.Message, "Ошибка",
    MessageBoxButton.OK, MessageBoxImage.Error);
}
}

```

```

// Метод для проверки корректности email адреса с использованием регулярного выражения
Ссылка: 1
private bool IsValidEmail(string emailAddress)
{
    return Regex.IsMatch(emailAddress, @"^[^@\s]+@^[^@\s]+\.[^@\s]+$");
}

// Метод для проверки корректности номера телефона с использованием регулярного выражения
private bool IsValidPhone(string phoneNumber)
{
    return Regex.IsMatch(phoneNumber, @"^\+\d{11}$");
}

```

Рисунок 14 – Проверка данных

Окно SaleWindow

– Проектирование интерфейса

Я спланировала окно для работы с продажами:

- **Цели:** Ввод названия продукта, даты и количества с привязкой к партнёру.
- **Структура:** Поля ввода вертикально, кнопки «Сохранить» и «Отменить» внизу.
- **Функциональность:** Добавление и редактирование продаж.

Прототипирование интерфейса

В SaleWindow.xaml я создала базовую версию:

<Grid> с <StackPanel>:

<Grid>

<StackPanel Margin="10">

<TextBlock Text="Наименование продукта"/>

<TextBox x:Name="ProductNameTextBox"/>

```

        <TextBlock Text="Дата продажи"/>
        <DatePicker x:Name="SaleDatePicker"/>
    </StackPanel> </Grid>

```

Добавила остальные поля и кнопки, проверил запуск.

– Стилизация интерфейса

Я улучшила внешний вид:

Окно: Height="300" Width="400" ResizeMode="CanMinimize"
WindowStartupLocation="CenterOwner".

Поля: Отступы для <TextBlock> и <TextBox>/DatePicker:

```

<TextBlock Text="Наименование продукта" Margin="0,0,0,5"/>
<TextBox x:Name="ProductNameTextBox" Margin="0,0,0,10"/>

```

Кнопки:

```

<StackPanel Orientation="Horizontal" HorizontalAlignment="Right"> с

```

отступом между кнопками:

```

<Button Content="Сохранить" Click="SaveButton_Click"/>
<Button Content="Отменить" Click="CancelButton_Click"
Margin="10,0,0,0"/>

```

В итоге получилось окно:

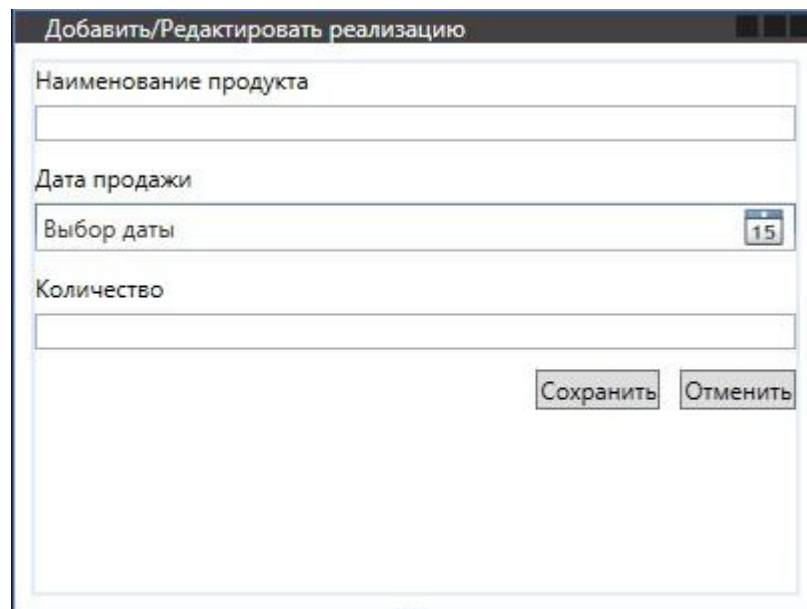


Рисунок 15 – Окно SaleWindow.xaml

Реализация методов в SaleWindow.xaml.cs

Я написала код:

```
public partial class SaleWindow : Window
{
    private readonly ApplicationContext _databaseContext; // Контекст базы данных
    private Sale _currentSale; // Текущая продажа (для редактирования или добавления)
    private bool _isEditing; // Флаг, указывающий, редактируется ли продажа
    private readonly Service _partnerManager; // Сервис для работы с данными
    private Partner _currentPartner; // Партнёр, с которым связана продажа

    // Конструктор, который принимает контекст базы данных, продажу и партнёра
    // Ссылка: 2
    public SaleWindow(ApplicationContext context, Sale sale = null, Partner partner = null)
    {
        InitializeComponent();
        _databaseContext = context;
        _partnerManager = new Service();
        _currentPartner = partner ?? throw new ArgumentNullException(nameof(partner), "Партнер не выбран.");

        // Если продажа передана, это режим редактирования
        if (sale != null)
        {
            _isEditing = true;
            _currentSale = sale;
            Title = "Редактирование продукта"; // Заголовок окна для редактирования
            LoadSaleDetails(); // Загружаем данные для редактирования
        }
        else
        {
            _isEditing = false;
            Title = "Добавление продукта"; // Заголовок окна для добавления новой продажи
            _currentSale = new Sale(); // Создаём новый объект продажи
        }
    }
}
```

Рисунок 16 – Инициализация

```

// Метод для обработки кнопки сохранения данных продажи
Ссылка 1
private void SaveButton_Click(object sender, RoutedEventArgs e)
{
    try
    {
        // Проверка, что наименование продукта не пустое
        if (string.IsNullOrWhiteSpace(ProductNameTextBox.Text))
        {
            MessageBox.Show("Введите корректное наименование продукта.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }

        // Проверка, что количество положительное число
        if (!int.TryParse(QuantityTextBox.Text, out int saleQuantity) || saleQuantity <= 0)
        {
            MessageBox.Show("Количество должно быть положительным числом.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }

        // Проверка, что дата продажи выбрана и не в будущем
        if (SaleDatePicker.SelectedDate == null)
        {
            MessageBox.Show("Выберите дату продажи.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }

        if (SaleDatePicker.SelectedDate > DateTime.Today)
        {
            MessageBox.Show("Дата продажи не может быть в будущем.", "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }

        // Сохранение данных в объект продажи
        _currentSale.ProductName = ProductNameTextBox.Text;
        _currentSale.Quantity = saleQuantity;
        _currentSale.Date = SaleDatePicker.SelectedDate.Value;
        _currentSale.PartnerId = _currentPartner.Id;

        // Если редактируем существующую продажу
        if (_isEditing)
        {
            _partnerManager.UpdateSale(_dbContext, _currentSale); // Обновляем продажу в базе
        }
        else
        {
            _partnerManager.AddSale(_dbContext, _currentSale); // Добавляем новую продажу
        }

        _partnerManager.SaveChanges(_dbContext); // Сохраняем изменения в базе данных
        DialogResult = true; // Закрытие окна с результатом успешного сохранения
        Close();
    }
    catch (Exception error)
    {
        MessageBox.Show($"Произошла ошибка: {error.Message}", "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error); // Обработка ошибок
    }
}

```

Рисунок 16 – Сохранение

Сопровождение системы

Сопровождение системы на данном этапе заключается в поддержании её работоспособности и устранении проблем, которые могут возникнуть при использовании. Я слежу за тем, чтобы приложение стабильно работало, подключалось к базе данных и выполняло свои основные функции. Если пользователи (например, заказчик) сообщают об ошибках, я анализирую их, воспроизвожу в Visual Studio и исправляю, проверяя изменения через существующие тесты.

Система может быть улучшена в процессе сопровождения, если это потребуется. Например, при обнаружении ошибок я не только устраняю их, но и, при необходимости, добавляю дополнительные проверки или улучшаю код для предотвращения подобных ситуаций в будущем. Также, если заказчик просит новые функции или изменения (например, фильтр в списке партнёров или новый формат отображения данных), я дорабатываю приложение, основываясь на его запросах. Все улучшения внедряются постепенно, с учётом текущей структуры системы и потребностей пользователя.

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества;

– **Проектирование и разработка кода**

Для создания кода применяются подходы, такие как разделение программы на независимые модули, каждый из которых решает свою задачу, что облегчает тестирование и поддержку; построение структурированного кода через классы и объекты с использованием принципов наследования, инкапсуляции и полиморфизма; акцент на чистые функции и неизменяемые данные для повышения надежности; а также внедрение проверенных шаблонов проектирования, таких как Singleton или Observer, для улучшения читаемости.

– **Обеспечение качества кода**

Качество кода достигается через совместную проверку кода разработчиками, автоматизированный анализ с использованием инструментов вроде SonarQube или ESLint для выявления ошибок и уязвимостей, а также многоуровневое тестирование – от проверки отдельных функций до оценки работы всей системы в целом.

– **Управление процессом разработки**

Процесс разработки оптимизируется за счет применения гибких методологий, таких как Scrum, автоматизации интеграции и развертывания

кода, а также соблюдения стандартов кодирования, что ускоряет работу и повышает стабильность продукта.

- **Улучшение и оптимизация кода**

Для повышения качества код подвергается реструктуризации и анализу производительности, что позволяет устранить слабые места и сделать его более удобным для сопровождения.

- **Документация и коммуникация**

Сопровождение кода упрощается за счет создания понятной документации и использования систем управления задачами, таких как Jira, для отслеживания требований и прогресса.

- **Инструменты и технологии**

Контроль версий осуществляется с помощью систем вроде Git, а процесс разработки ускоряется благодаря интегрированным средам (IDE) и готовым библиотекам, что повышает эффективность работы.

- **Управление требованиями**

Четкое понимание функциональности достигается через анализ требований и их описание в виде пользовательских историй, что помогает точно реализовать ожидания заказчика.

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

Процесс инспекции программного кода направлен на обеспечение его соответствия установленным стандартам и лучшим практикам разработки. Инспекция позволяет выявить ошибки, повысить читаемость кода, улучшить производительность и предотвратить потенциальные уязвимости.

Виды инспекции кода

В зависимости от цели проверки применяются следующие методы инспекции:

Статический анализ кода:

- выполняется без запуска программы;

- осуществляет проверку синтаксиса, стиля кодирования, обнаружение потенциальных ошибок и уязвимостей;
- позволяет выявить нарушения архитектурных стандартов и использовать рекомендации по их исправлению.

Ревью кода:

- выполняется вручную разработчиками или техническими лидерами команды;
- проверяет соответствие кода стандартам, логическую правильность, соблюдение принципов проектирования (SOLID, DRY, KISS);
- позволяет выявить потенциальные проблемы производительности, нарушения принципов архитектуры, сложность сопровождения кода.

Анализ зависимости:

- исследует используемые библиотеки и фреймворки;
- позволяет обнаружить устаревшие или небезопасные зависимости, потенциальные проблемы производительности.

Актуальные стандарты кодирования

Инспекция кода выполняется в соответствии с установленными стандартами, которые определяют правила написания, архитектурные принципы и соглашения о стиле кодирования:

1. Microsoft C# Coding Conventions

- определяет форматирование кода, отступы, именование переменных и методов;
- требует явного указания модификаторов доступа, использования `var` только там, где это обосновано, и правильного форматирования выражений LINQ;

2. .NET Runtime Coding Guidelines

- рекомендации по управлению памятью, обработке исключений, использованию асинхронного программирования;
- регламентирует использование `IDisposable` для работы с ресурсами.

3. SOLID-принципы

описывает принципы проектирования программного обеспечения для повышения читаемости и масштабируемости.

4. Entity Framework Best Practices

- рекомендации по оптимальному использованию ORM Entity Framework;
- включает предотвращение утечек памяти, оптимизацию запросов и корректное управление контекстом базы данных.

Инструменты инспекции кода

Для автоматизации проверки кода используются специализированные инструменты. Рассмотрим два наиболее популярных решения:

1. Roslyn Analyzers Roslyn – это компилятор C# и VB.NET, встроенный в .NET, который поддерживает мощный механизм анализа кода. Roslyn Analyzers позволяют выявлять ошибки, предупреждения и предложения по улучшению кода.

Основные возможности:

- автоматическое обнаружение нарушений стандартов кодирования;
- проверка корректности использования API .NET;
- анализ потенциальных ошибок и уязвимостей в коде;
- интеграция с Visual Studio и поддержка кастомных правил анализа;

2. SonarQube SonarQube – это мощный инструмент для статического анализа кода, используемый для контроля качества программных проектов.

Основные возможности:

- проверка на соответствие стандартам кодирования (например, Microsoft C# Coding Conventions);
- обнаружение потенциальных ошибок, утечек памяти и уязвимостей;
- анализ дублирования кода и выявление сложных для понимания конструкций;
- глубокая интеграция с CI/CD (Jenkins, Azure DevOps, GitHub Actions).

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств;

Для начала я запустила мое приложение и попробовала создать несколько партнеров (см. Рис. 17-20).

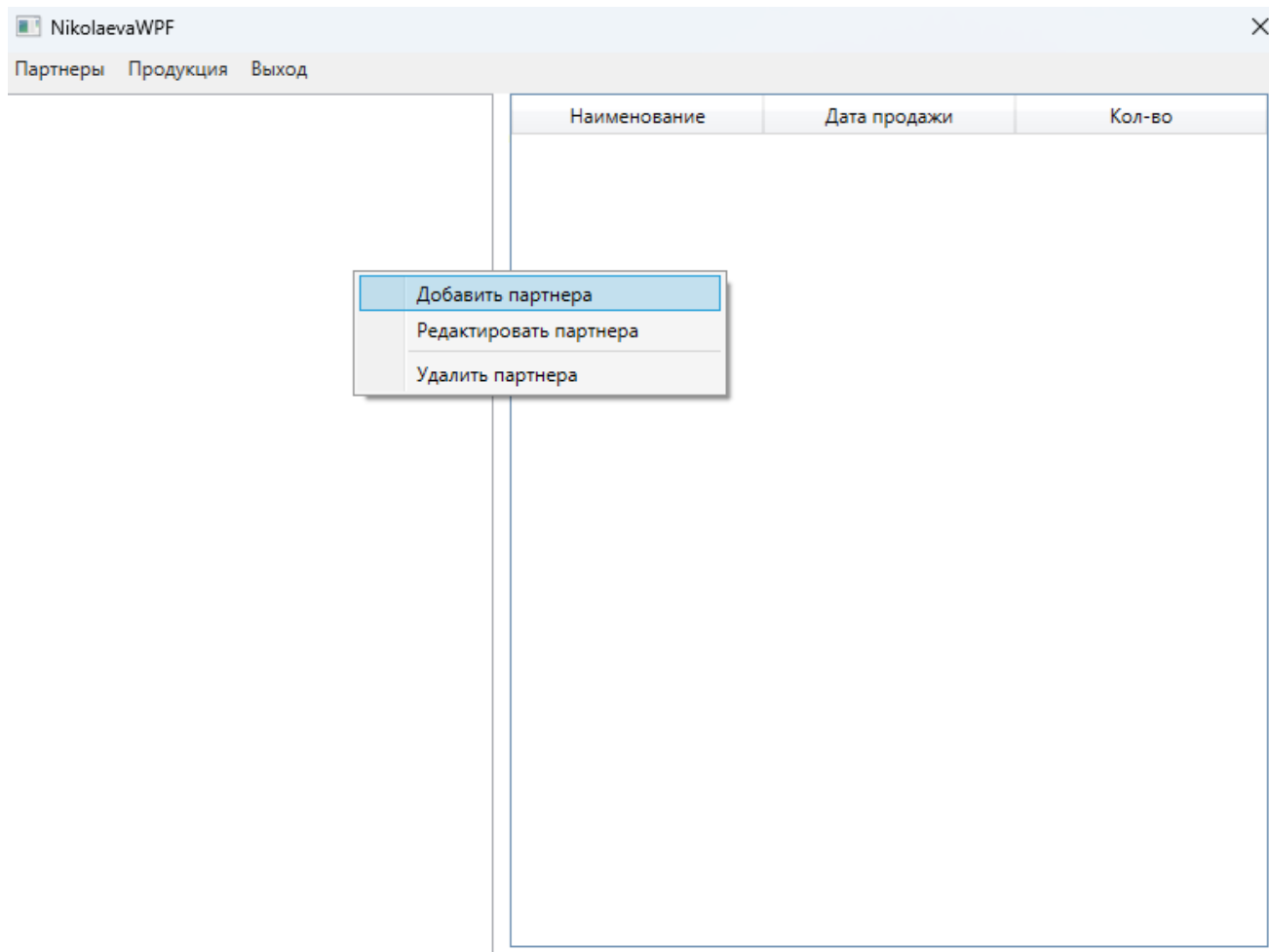


Рисунок 17 – Процесс создания Партнеров

Добавление партнера

Тип партнера
ООО

Название
Тестовая

Директор
Директоров.В.В

Email
director@mail.ru

Телефон
+79553211122

Юридический адрес
Г.Пермь ул.Лермонтова 52

Рейтинг
5

Сохранить Отменить

Рисунок 18 – Окно создания Партнера

Добавление партнера

Тип партнера
ПАО

Название
Проверка

Директор
Паршаков.Д.В

Email
dirpr@yandex.ru

Телефон
+79423332211

Юридический адрес
Г.Пермь ул.Ломоносова 67

Рейтинг
8

Сохранить Отменить

Рисунок 19 – Окно создания Партнера

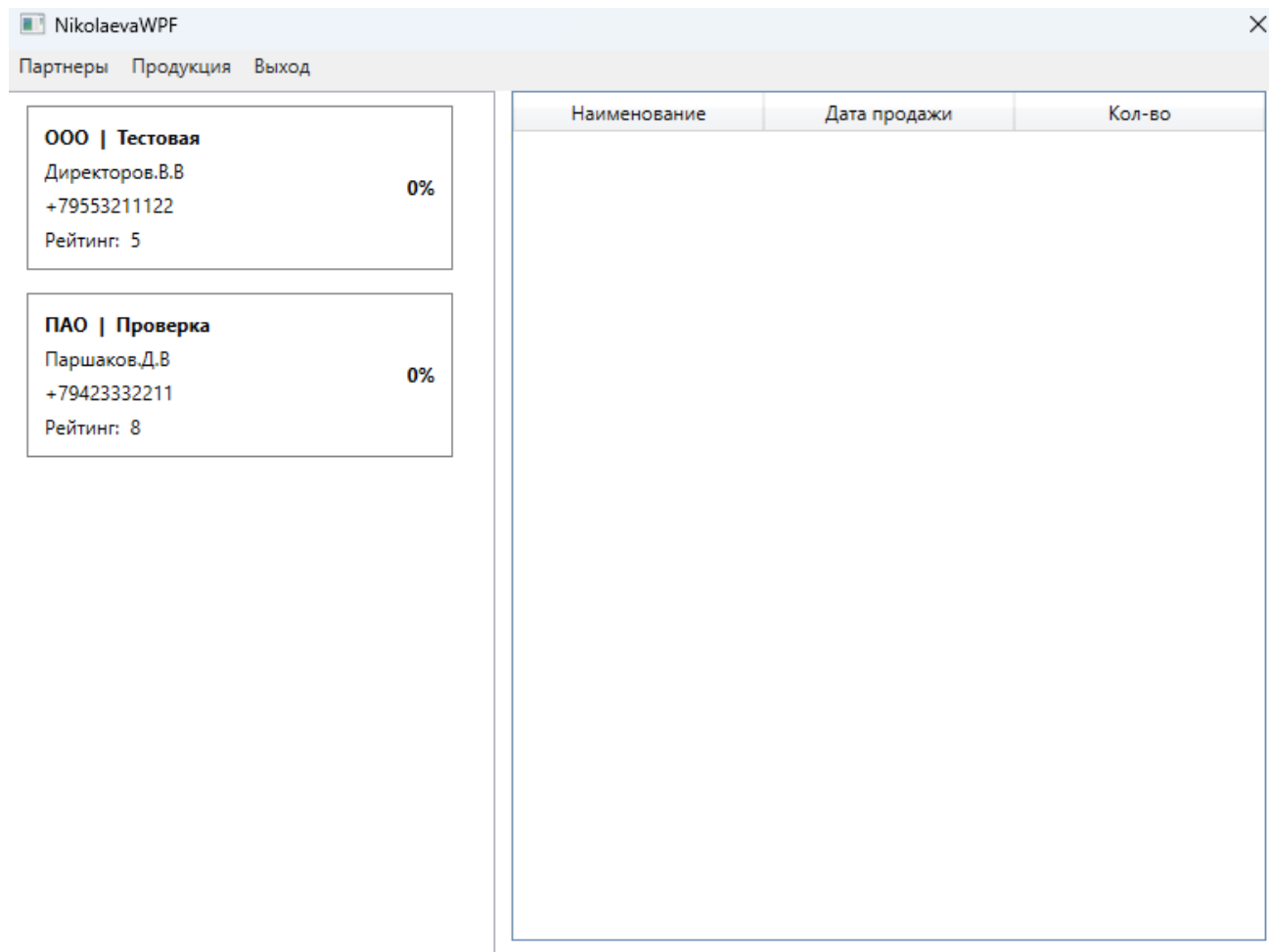


Рисунок 20 – Главное окно

После того как мы создали партнеров попробуем одного отредактировать, а второго удалить (см. Рис. 21-23).

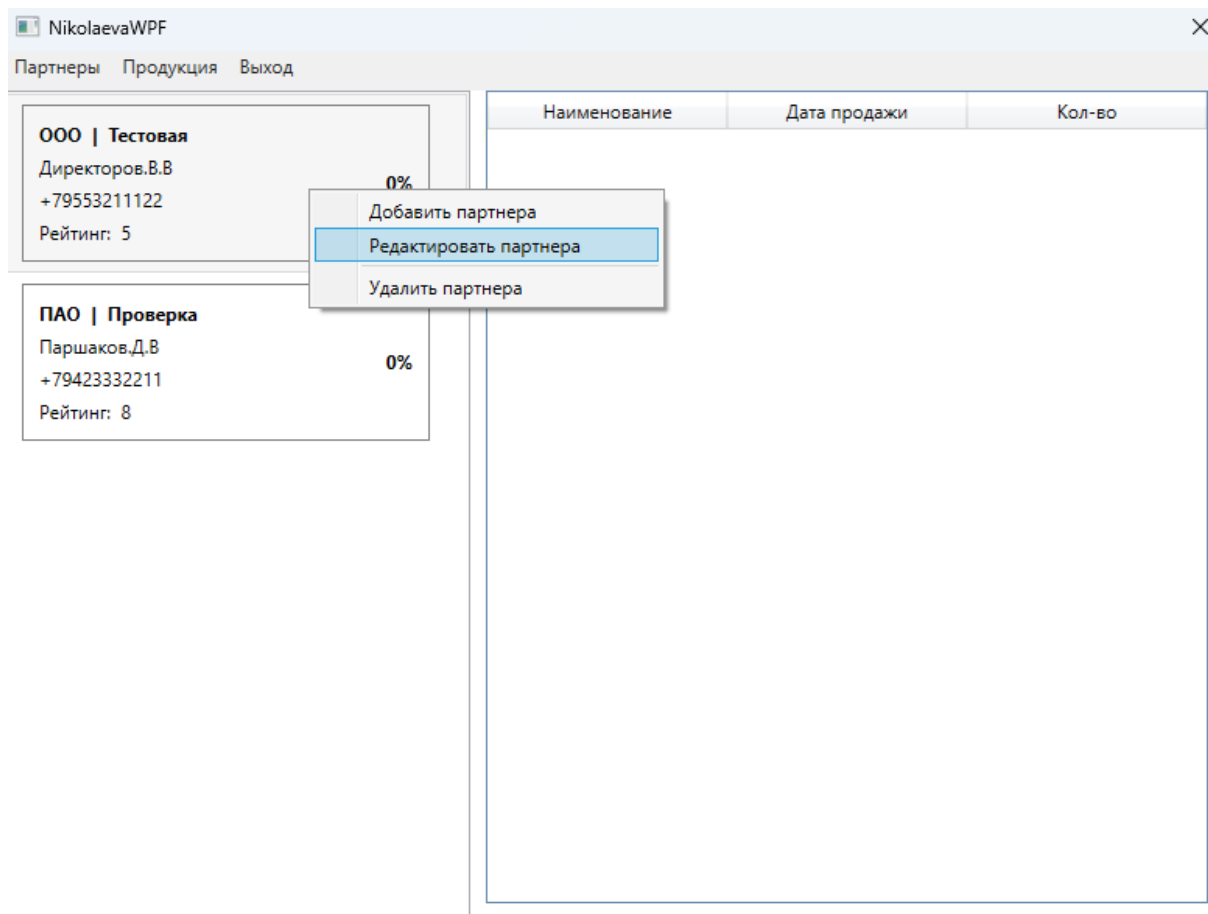


Рисунок 21 – Процесс Редактирования партнера

Редактирование партнера

Тип партнера
ООО

Название
Тестовая1

Директор
Директоров.В.В

Email
director@mail.ru

Телефон
+79553211122

Юридический адрес
Г.Пермь ул.Лермонтова 52

Рейтинг
5

Сохранить Отменить

Рисунок 22 – Окно редактирования партнера

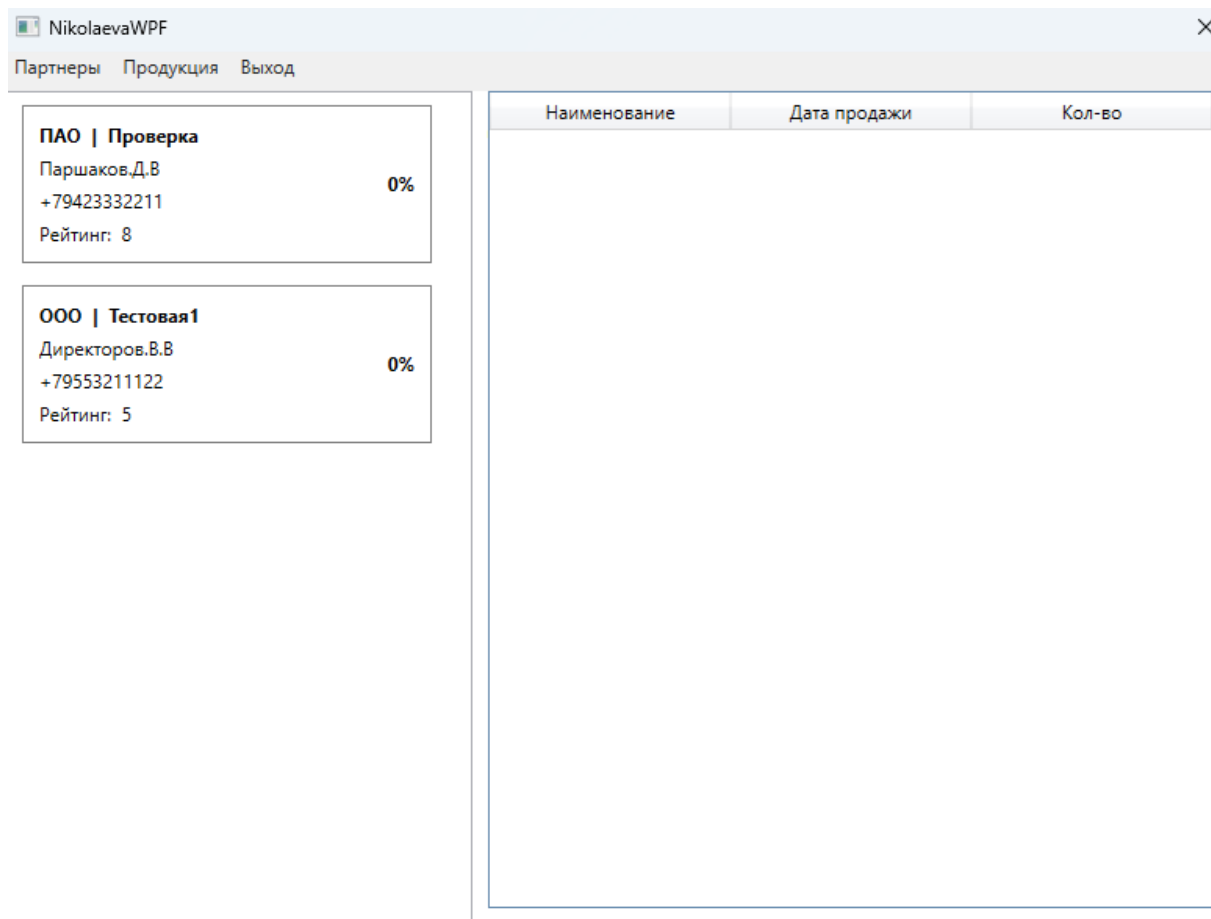


Рисунок 23 – Окно с измененными данными о партнере

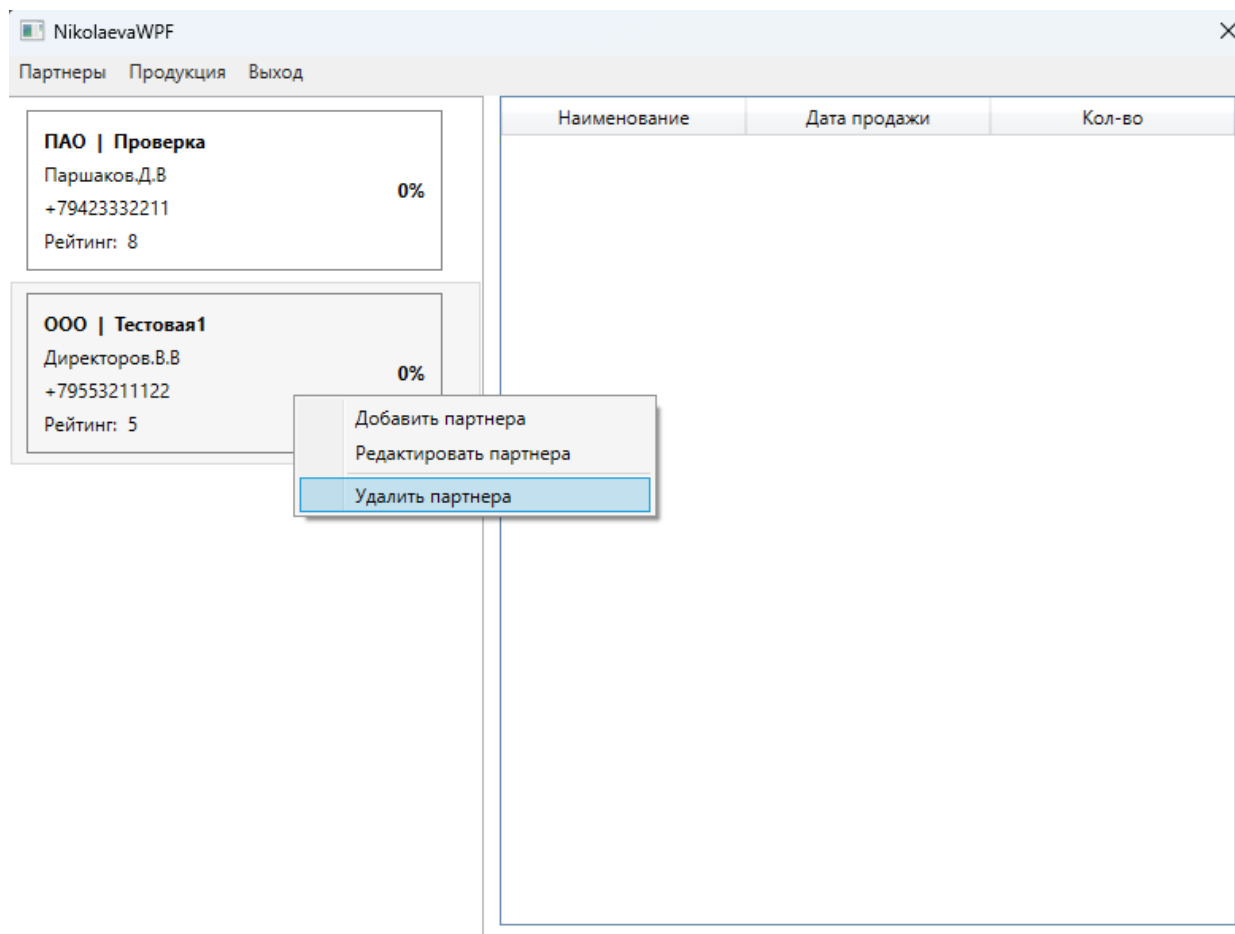


Рисунок 24 – Процесс удаления партнера

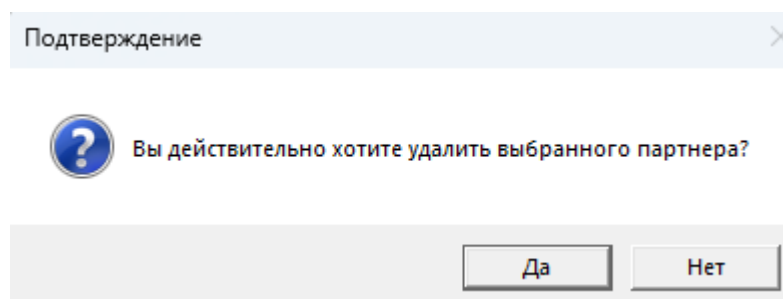


Рисунок 25 – Процесс удаления партнера

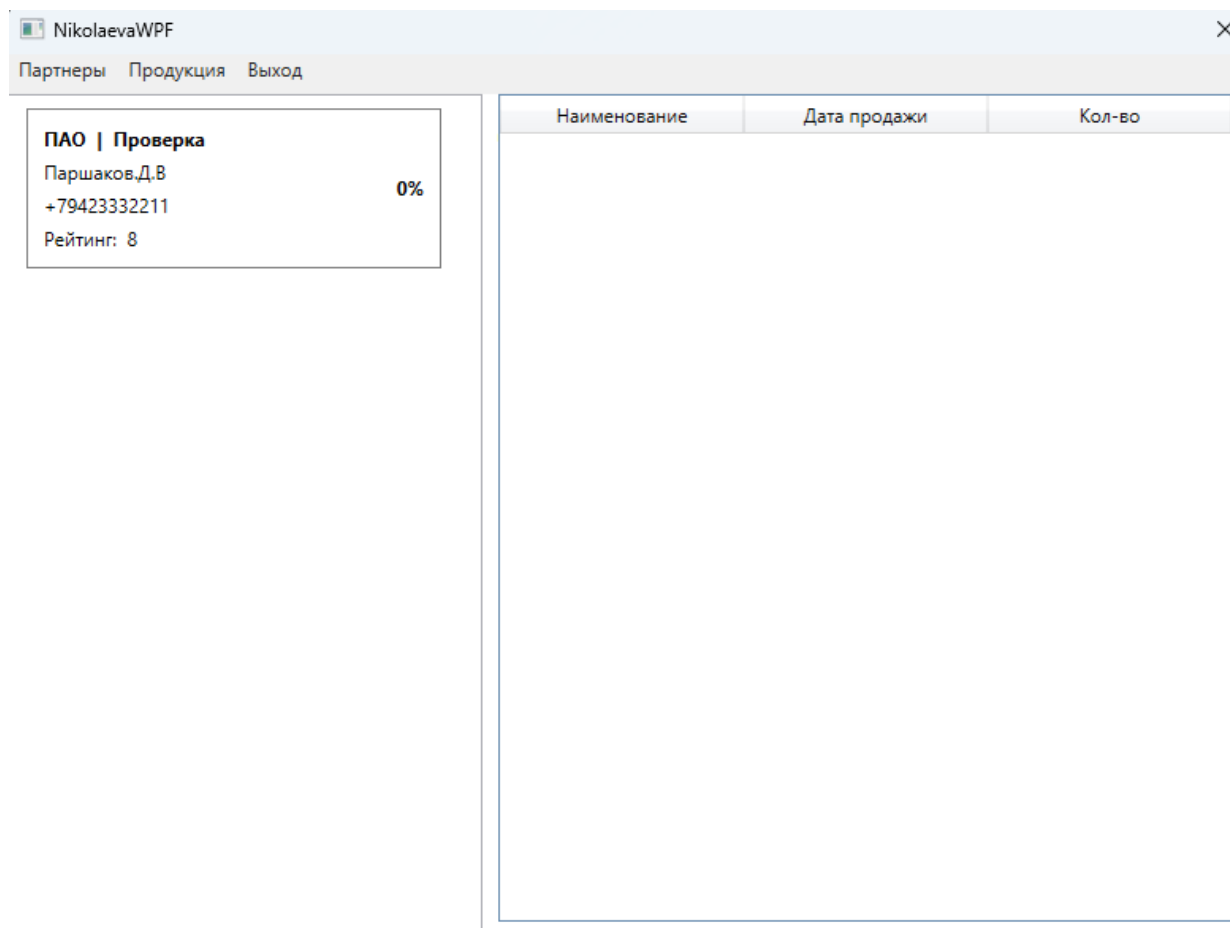


Рисунок 26 – Итог удаления партнера

После проделанных действий производим манипуляции с продажами, начнём с добавления (см. Рис. 27-29).

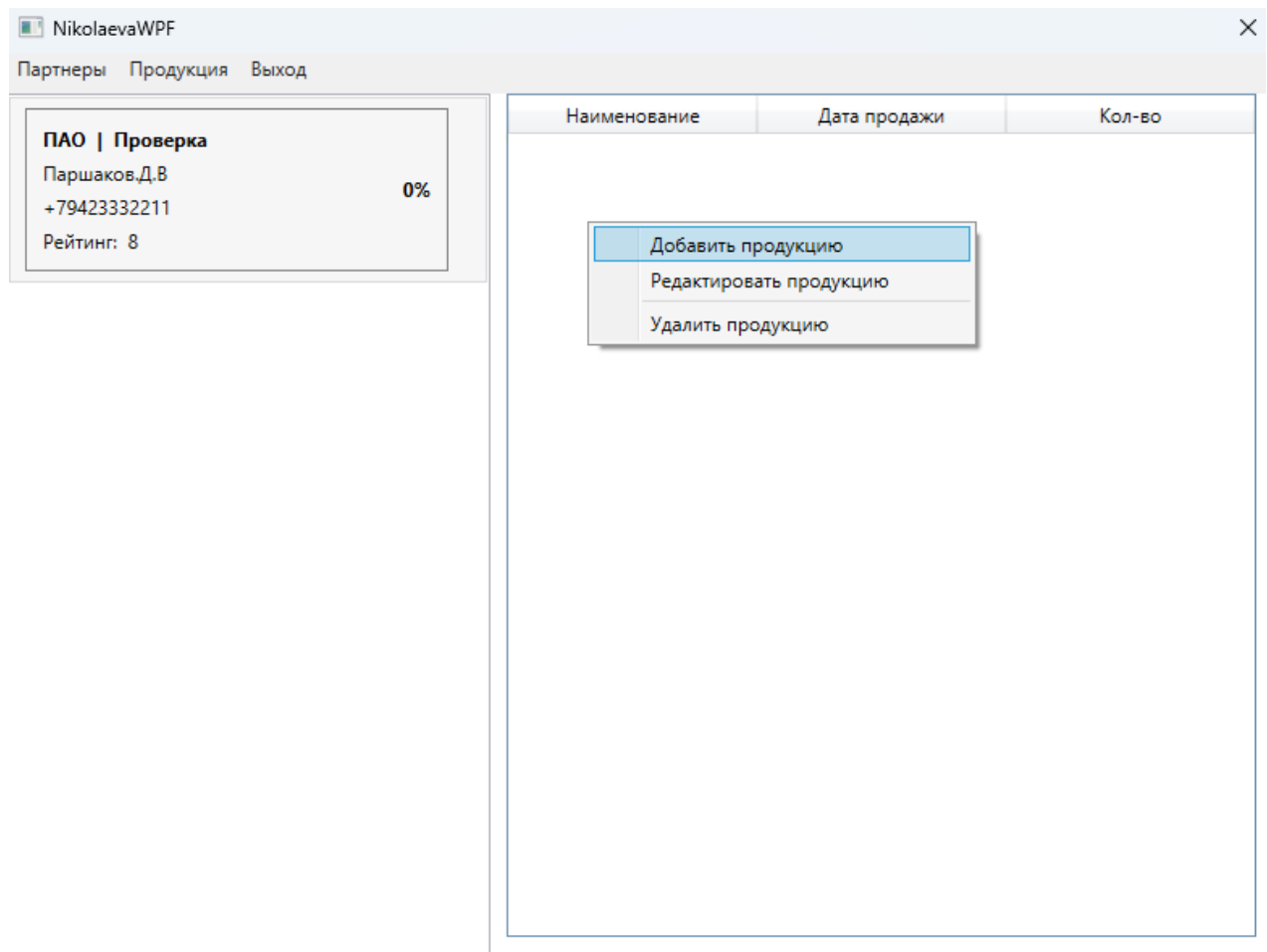


Рисунок 27 – Процесс добавления продукции

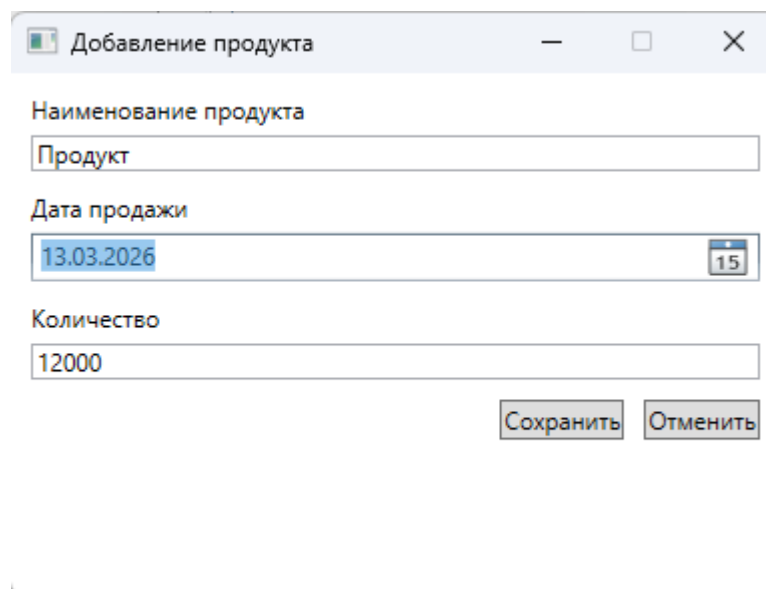


Рисунок 28 – Процесс добавления продукции

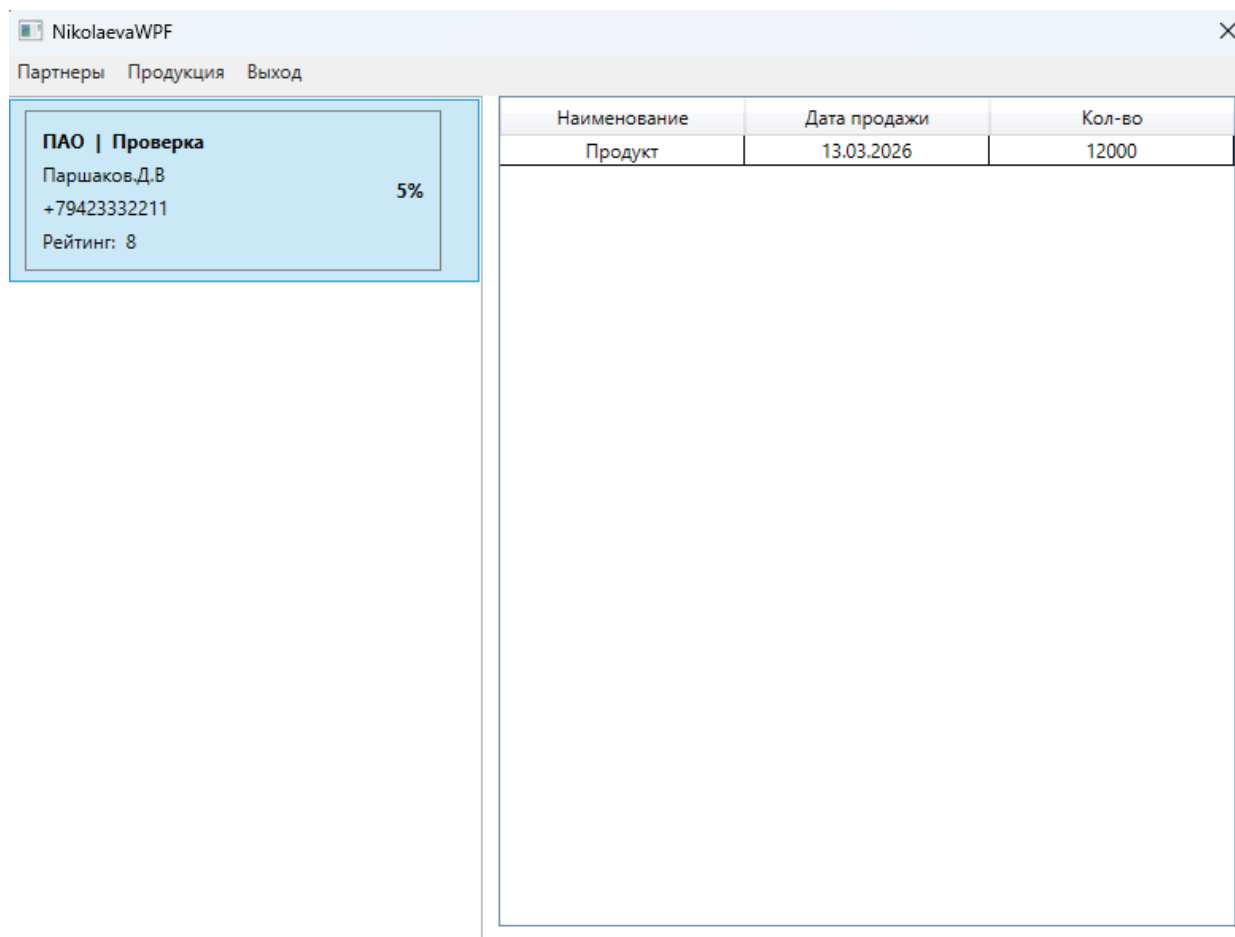


Рисунок 29 – Итог добавления продукции

Также проверяем редактирование (см. Рис. 30-32).

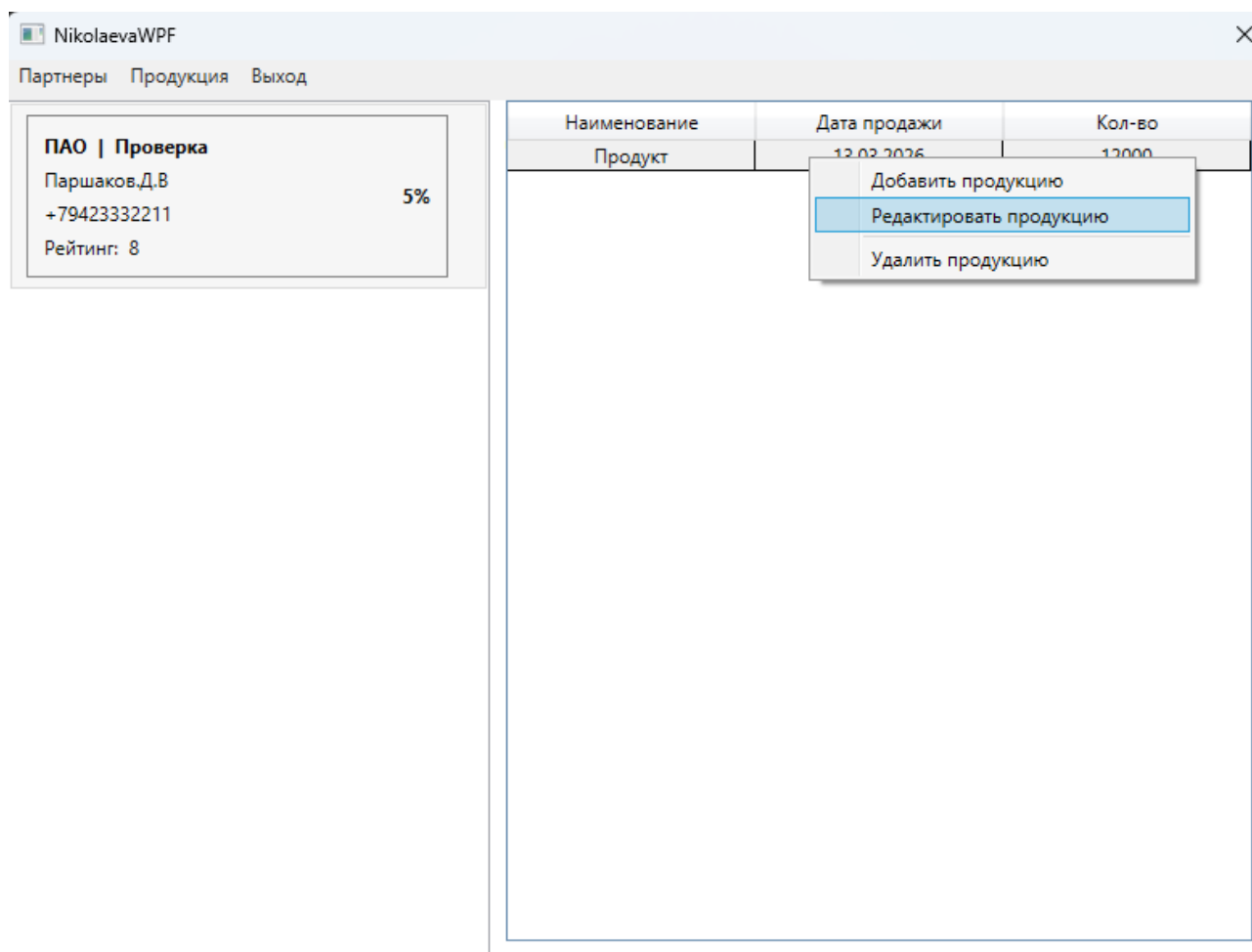


Рисунок 30 – Процесс редактирования продукции

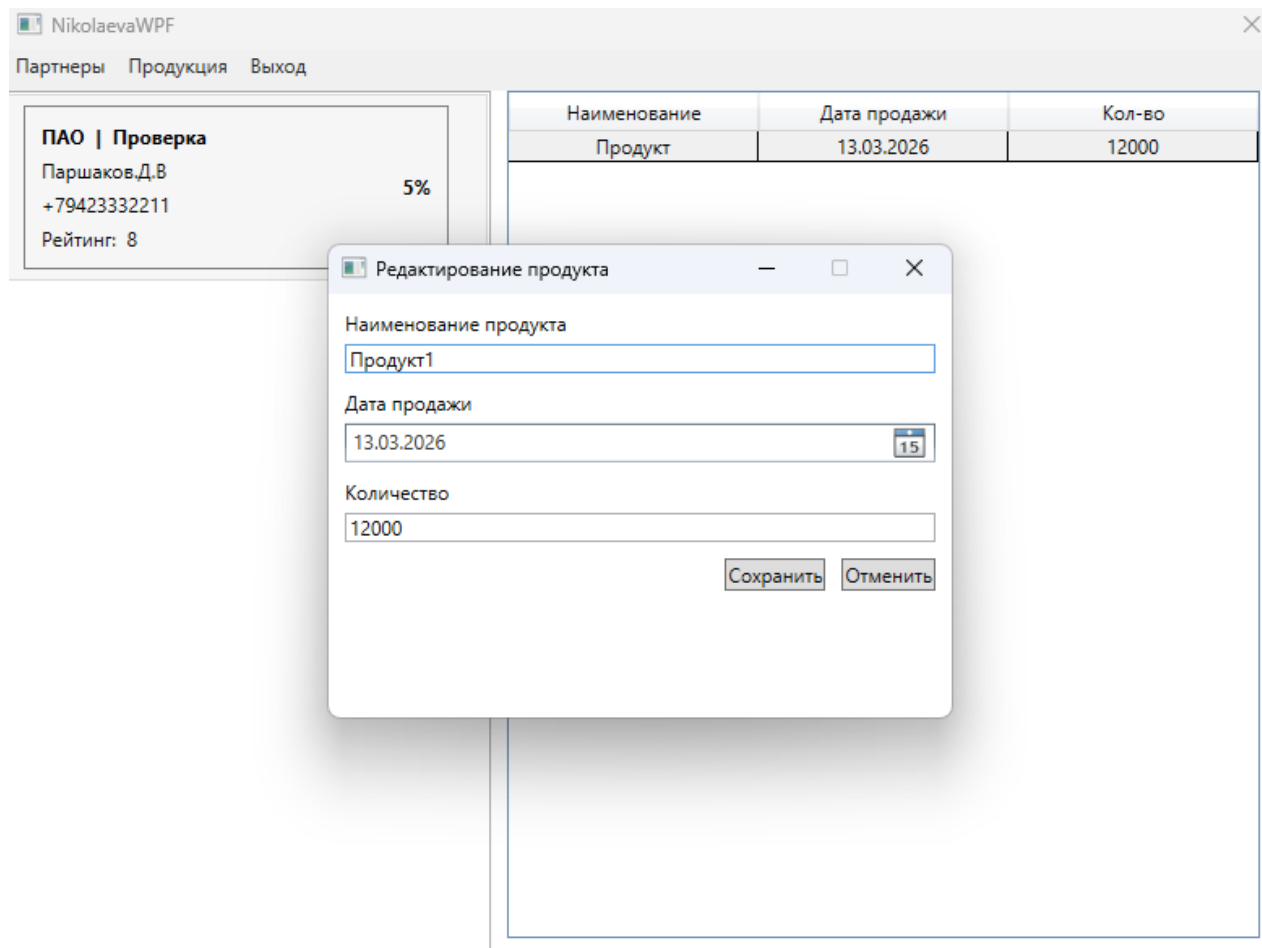


Рисунок 31 – Процесс редактирования продукции

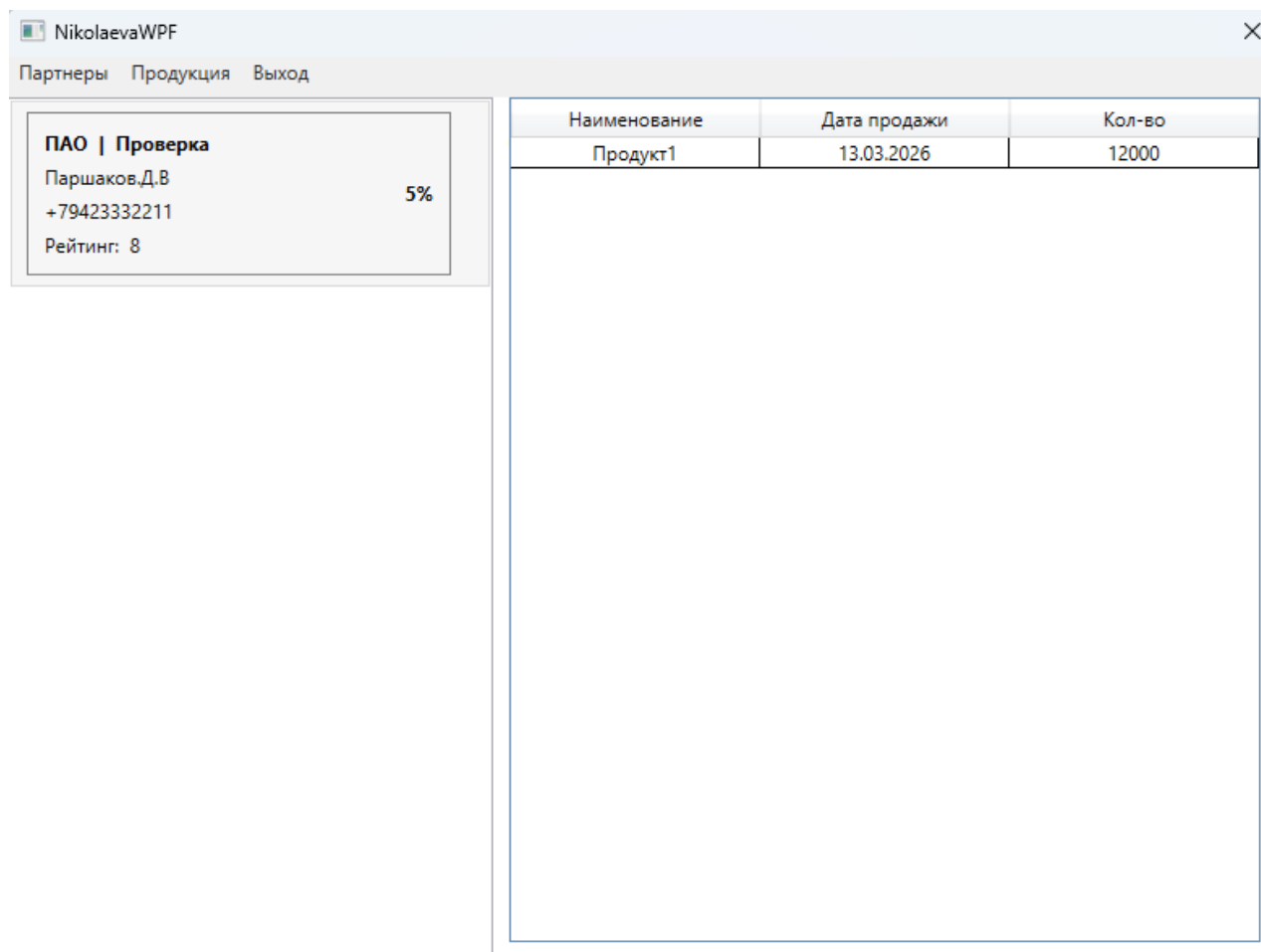


Рисунок 32 – Итог редактирования продукции

И итогом стала проверка удаления, которую я произвела на специально созданной продаже (см. Рис. 33-35).

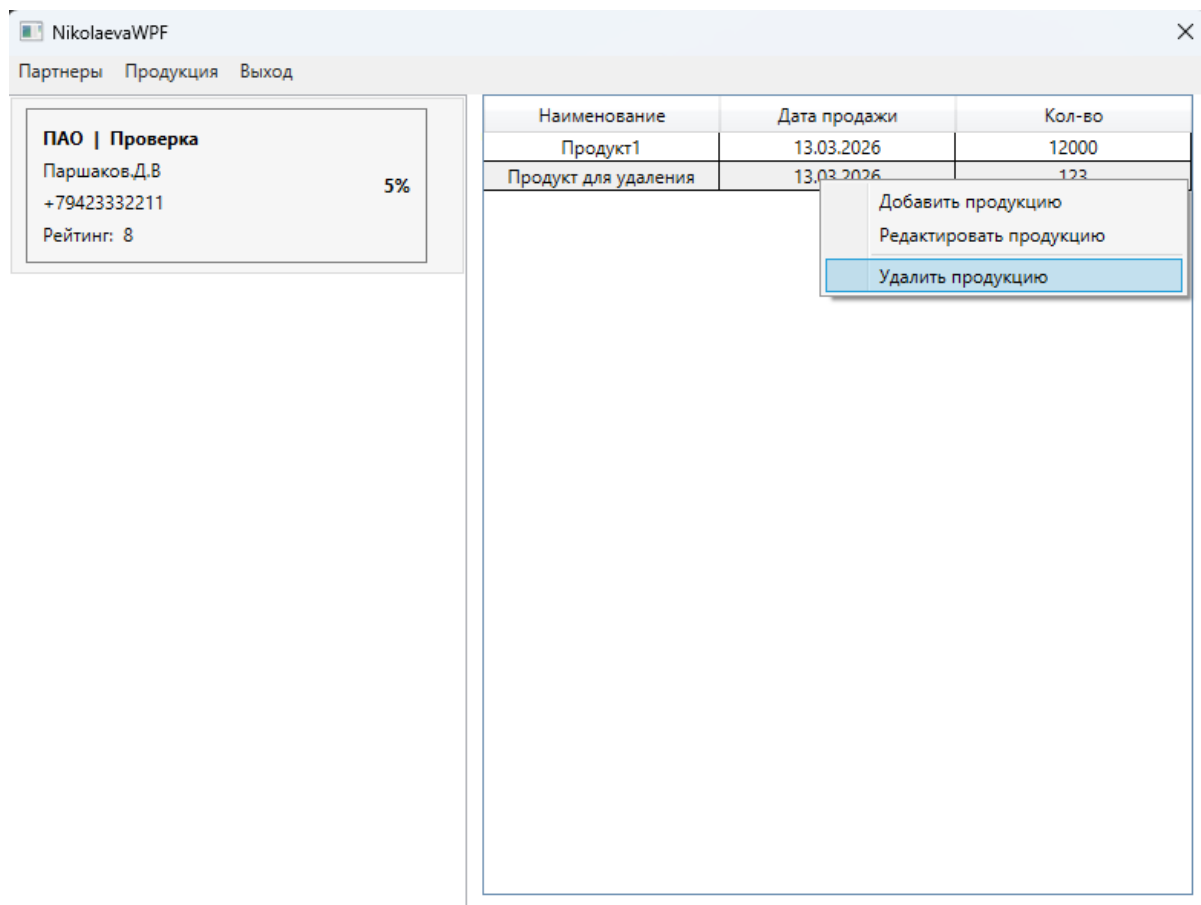


Рисунок 33 – Процесс удаления продукции

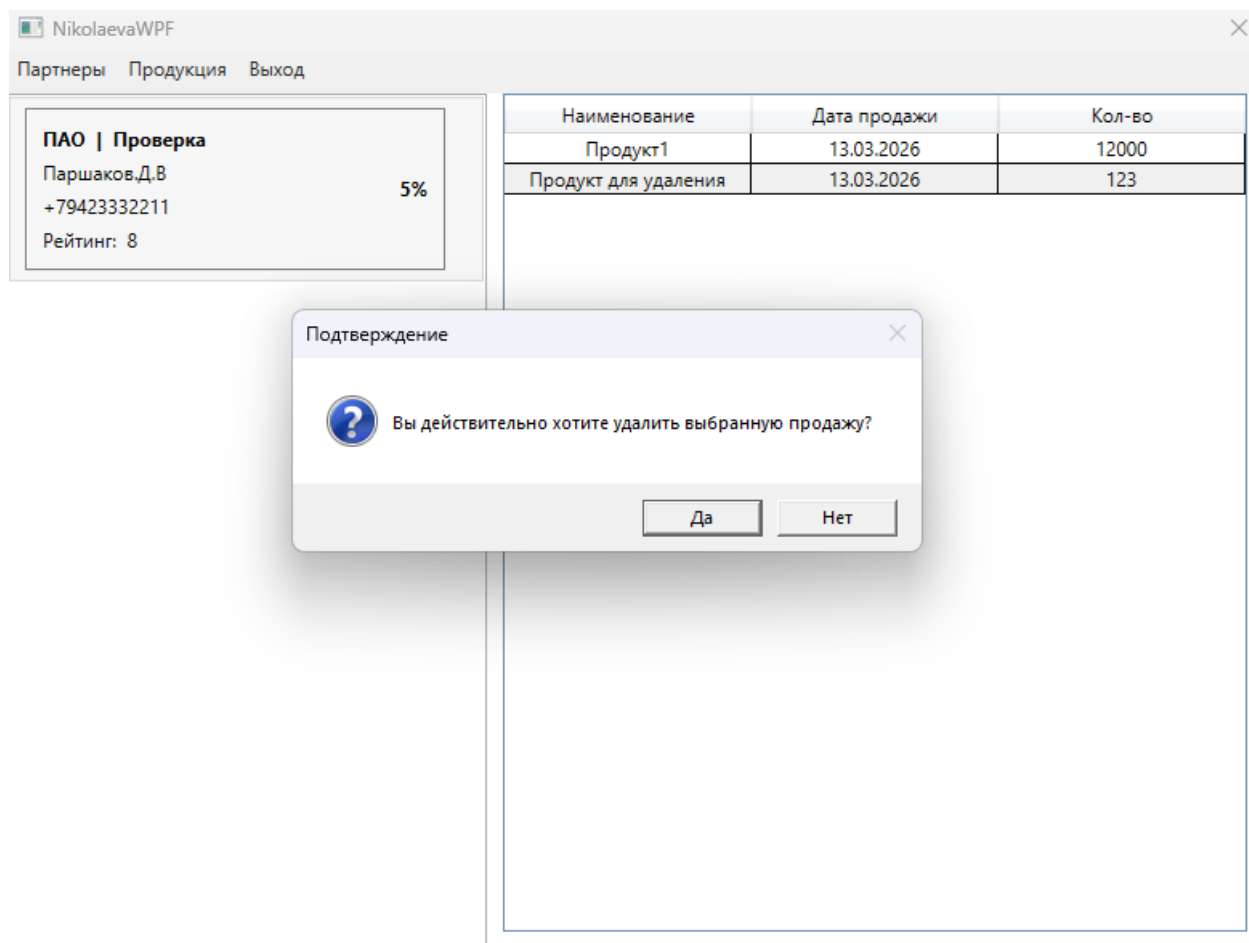


Рисунок 34 – Процесс удаления продукции

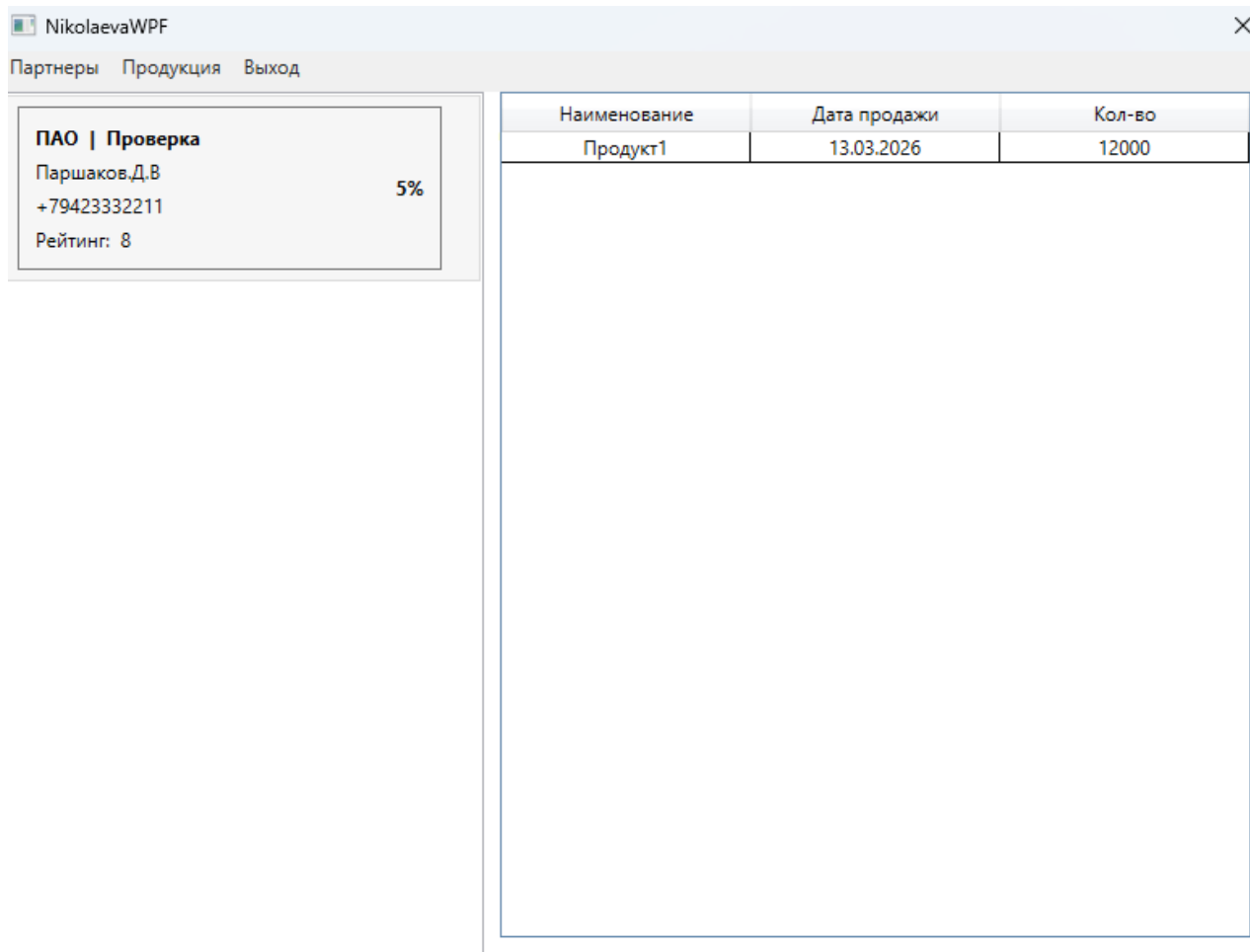


Рисунок 35 – Итог удаления продукции

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения;

Тестирование программного обеспечения (ПО) – это процесс оценки и проверки работы приложения с целью выявления ошибок, обеспечения качества и соответствия требованиям.

Основные этапы:

1. Анализ требований:

изучение функциональных и нефункциональных требований;
определение критических участков системы.

2. Планирование тестирования:

– определение стратегии тестирования;

- выбор инструментов и методов; подготовка тестовых данных.

3. Разработка тест-кейсов:

- определение сценариев и условий тестирования;
- написание юнит-тестов, интеграционных и функциональных тестов.

4. Выполнение тестов:

автоматическое и ручное выполнение тестов; логирование результатов и фиксация ошибок.

5. Анализ результатов:

- оценка покрытия кода тестами;
- анализ ошибок и их устранение.

6. Ретест и регрессионное тестирование:

- повторное тестирование после исправлений;
- проверка влияния изменений на другие части системы.

7. Документирование и отчетность:

- описание найденных дефектов;
- подготовка отчетов о тестировании.

Unit-тесты:

Подробно можно ознакомиться в пункте 3.1.2.4.

3.1.7 Интегрирование программных модулей;

1. Планирование интеграции

- Цель: проверить логику библиотеки NikolaevaLib через тесты в NikolaevaTests, затем подключить её к NikolaevaWPF для отображения данных в интерфейсе;
- Стратегия: сначала интеграция с тестами, затем с WPF, чтобы убедиться в корректности ядра перед подключением интерфейса;
- Роли проектов: NikolaevaLib – ядро с логикой и моделями, NikolaevaTests – проверка функциональности, NikolaevaWPF – интерфейс.

2. Подготовка библиотеки классов (NikolaevaLib)

Состав:

- ApplicationContext.cs – отвечает за подключение к базе данных (например, настройки соединения);
- Discount.cs, Partner.cs, Sale.cs – модели таблиц БД, описывающие скидки, партнеров и продажи;
- PartnerView.cs – возможно, вспомогательная структура для представления данных о партнерах;
- Service.cs – содержит методы для работы с данными (например, получение списка партнеров, добавление продажи).

Действия: проверяем, что все зависимости (например, драйвер БД) установлены, и библиотека собирается без ошибок.

3. Интеграция с проектом модульных тестов (NikolaevaTests)

Устанавливаем связь: добавляем в NikolaevaTests ссылку на NikolaevaLib, чтобы тесты могли использовать её функциональность.

Настраиваем тесты:

- В NikolaevaTests.cs создаем сценарии для проверки методов из Service.cs (например, тест на получение списка партнеров или создание скидки);
- Если нужно избежать работы с реальной БД, используем имитацию подключения через ApplicationContext.cs.

Проверяем:

Запускаем тесты, убеждаемся, что методы библиотеки работают корректно, и устраняем ошибки, если они есть.

4. Интеграция с WPF-проектом (NikolaevaWPF)

Устанавливаем связь: добавляем в NikolaevaWPF ссылку на NikolaevaLib для доступа к моделям и сервису.

Настраиваем интерфейс:

- В MainWindow.xaml создаем основное окно, которое может служить точкой входа или панелью навигации;
- В PartnerWindow.xaml настраиваем отображение данных о партнерах (например, список или таблицу), используя сервис из Service.cs и модели Partner.cs или PartnerView.cs;
- В SaleWindow.xaml настраиваем интерфейс для работы с продажами, связывая его с Sale.cs и методами сервиса;
- Создаем промежуточный слой (например, ViewModel) для передачи данных из Service.cs в окна.

Проверяем:

Запускаем приложение, чтобы данные (например, список партнеров или продажи) отображались в окнах.

5. Сборка и тестирование системы

Собираем решение целиком, проверяем, что все проекты компилируются без ошибок.

Проводим интеграционное тестирование:

- Убеждаемся, что тесты в NikolaevaTests.cs подтверждают работу методов из Service.cs;
- Проверяем, что окна PartnerWindow.xaml и SaleWindow.xaml корректно отображают данные из БД через NikolaevaLib.

Исправляем проблемы:

Если данные не доходят до интерфейса или тесты не проходят, отлаживаем взаимодействие.

6. Финализация

- Настраиваем конфигурацию: например, выносим настройки подключения из ApplicationContext.cs в отдельный файл, доступный всем проектам;
- Проводим финальную проверку: запускаем тесты и приложение, чтобы убедиться, что всё работает как единая система;

- Готовим к использованию: проект готов к дальнейшей разработке или развертыванию.

3.1.8. Верификация и аттестация программного обеспечения

Процесс проверки и сертификации программного обеспечения является важным этапом разработки, целью которого является подтверждение корректности, стабильности, а также соответствия требованиям и стандартам качества. В моем проекте реализован комплексный подход, включающий как автоматизированные, так и ручные методы тестирования, анализ кода и документирование результатов.

Этапы проверки программного обеспечения:

1. Автоматизированное тестирование и анализ кода

На начальной стадии осуществляется проверка исходного кода для обеспечения надежности и минимизации ошибок. Применяются следующие методы:

- **Unit-тестирование с использованием MSTest** – позволяет проверить работоспособность отдельных модулей, не зависящих от внешних ресурсов, например, функций форматирования данных, расчета скидок и валидации входных данных;

- **Статический анализ кода с помощью ReSharper и Visual Studio** – помогает выявить потенциальные ошибки, нарушение принципов SOLID и неэффективные конструкции;

- **Автоматический запуск тестов в среде CI/CD** – обеспечивает оперативную проверку изменений в коде и предотвращает регрессионные ошибки.

2. Интеграционное тестирование

После успешного прохождения unit-тестов проводится интеграционное тестирование, которое включает:

- оценку взаимодействия пользовательского интерфейса с бизнес-логикой; проверку работы с базой данных (например, корректность SQL-запросов через Npgsql);

- тестирование передачи данных между окнами, например, при переходе между MainWindow и EditOrderWindow.

3. Ручное тестирование и анализ кода

Процесс проверки также охватывает соответствие стандартам кодирования и архитектуры. Включает:

- ручное тестирование интерфейса для проверки удобства использования и корректности отображения данных;
- код-ревью для выявления потенциальных проблем, соответствия стандартам и улучшения архитектуры;
- оценку безопасности, включая защиту от SQL-инъекций и утечек данных.

Этапы сертификации программного обеспечения:

1. Оценка соответствия требованиям

Цель сертификации – удостовериться, что функциональность соответствует техническому заданию и стандартам качества.

Проверяются:

- соответствие реализованных функций требованиям спецификации;
- корректность выполнения ключевых пользовательских сценариев; □
производительность и устойчивость системы.

2. Нагрузочное тестирование и анализ производительности

В рамках сертификации проводится стресс-тестирование для оценки устойчивости системы под высокой нагрузкой.

Основные задачи:

- оценка стабильности при большом количестве активных пользователей;
- измерение времени выполнения ключевых операций (например, загрузка списка заказов или выполнение SQL-запросов);
- выявление узких мест и оптимизация производительности.

3. Документирование результатов

На этапе сертификации формируется официальная документация, включая:

- протоколы тестирования;
- отчеты об обнаруженных ошибках и их исправлениях;
- заключение о соответствии требованиям.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики с 09 по 17 марта 2026 года мною были закреплены теоретические знания и приобретены практические навыки по специальности 09.02.07 «Информационные системы и программирование».

В ходе практики была выполнена интеграция программных модулей в единую систему для учёта партнёров и продаж. Разработка велась на языке C# в среде Visual Studio с использованием .NET 9.0, Entity Framework Core и СУБД PostgreSQL. Были созданы библиотека классов NikolaevaLib, модульные тесты и пользовательский интерфейс на WPF. Задачи практики выполнены в полном объёме, профессиональные компетенции освоены.

По окончании практики был получен практический опыт интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ Р ИСО/МЭК 9294-93. Информационная технология. Руководство по управлению документированием программного обеспечения.
2. ГОСТ Р ИСО/МЭК 9126-93. Информационная технология. Оценка программной продукции. характеристика качества и руководство по их применению.
3. ГОСТ Р ИСО/МЭК 9127-94. Системы обработки информации. Документация пользователя и информация на упаковке для потребительских программных пакетов.
4. ГОСТ Р ИСО/МЭК 8631-94. Информационная технология. Программные конструктивы и условные обозначения для их представления.
5. ГОСТ Р ИСО/МЭК 12119:2000. Информационная технология. Пакеты программных средств. Требования к качеству и испытания.
6. ГОСТ 19.781- 90 - Термины и определения
7. ГОСТ 19.701-90 - Схемы алгоритмов, программ данных и схем. Условные обозначения и правила выполнения
8. ГОСТ Р ИСО/МЭК 12207-2010 «Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств», Федорова Г. Н. Участие в интеграции программных модулей. Учебное пособие / Федорова Г.Н. – М.Ж Издательский дом «Академия», 2016 – 304с.
9. Федорова Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности: Учебное пособие. / Федорова Г.Н. - М.:КУРС,НИЦ ИНФРА-М,2016. 336с.<http://znanium.com/catalog.php?bookinfo=544732>
10. Оценка качества программного обеспечения: Практикум: Учебное пособие / Б.В.
11. Черников, Б.Е. Поклонов; Под ред. Б.В. Черникова - М.: ИД ФОРУМ: НИЦ Инфра-М, 2012. - 400 с. <http://znanium.com/bookread2.php?book=315269>
12. Гуриков С. Р. Введение в программирование на языке Visual C#: Учебное пособие / С.Р.Гуриков.-М.:Форум:НИЦИНФРА-М,2013.448с.<http://znanium.com/bookread2.php?book=232661>
13. Исаченко О.В. Программное обеспечение компьютерных сетей : учебное пособие. М. : ИНФРА-М, 2013. - 117 с. - (СПО).
14. Стандартизация, сертификация и управление качеством программного обеспечения: Учебное пособие / Ананьева Т.Н., Новикова Н.Г., Исаев Г.Н. - М.:НИЦ ИНФРА-М, 2016. - 232с <http://znanium.com/bookread2.php?book=541003>
15. Гагарина Л.Г. Технология разработки программного обеспечения: Учебное пособие

- / Л.Г. Гагарина, Б.Д. Виснадул, Е.В. Кокорева. М.: ООО "Научноиздательский центр ИНФРА-М", 2013. 400 с. URL: <http://znanium.com/catalog.php?bookinfo=389963>
16. Немцова Т. И. Программирование на языке высокого уровня. Программир. на языке С++[Электронный ресурс]: Уч. пос. / Т.И.Немцова и др.; Под ред. Л.Г.Гагариной. М.: ИД ФОРУМ: ИНФРА-М, 2012. 512 с. URL: <http://znanium.com/catalog.php?bookinfo=244875>;
17. Рудаков А.В. Технология разработки программных продуктов: Учеб.пособие для студ. сред. проф. образования. М.: Издательство «Академия», 2010. 208 с.
18. Черников Б.В. Управление качеством программного обеспечения[Электронный ресурс]: Учебник / Б.В. Черников. - М.: ИД ФОРУМ: ИНФРА-М, 2012. - 240 с.. URL: <http://znanium.com/catalog.php?bookinfo=256901>.
19. Эйдлина Г.М. Delphi: программирование в примерах и задачах[Электронный ресурс]: Практикум / Г.М. Эйдлина, К.А. Милорадов. М.: ИЦ РИОР: НИЦ Инфра- М, 2012. 116 с. URL: <http://znanium.com/catalog.php?bookinfo=31904651>

Дополнительные источники:

1. Гагарина Л.Г. Разработка и эксплуатация автоматизированных информационных систем[Электронный ресурс]: Учебное пособие / Л.Г. Гагарина. М.: ИД ФОРУМ: НИЦ Инфра-М, 2013.384с. URL: <http://znanium.com/catalog.php?bookinfo=368454>;
2. Голицына О. Л. Программное обеспечение[Электронный ресурс]: Учебное пособие / О.Л. Голицына, Т.Л. Партыка, И.И. Попов. - 3-е изд., перераб.и доп. М.: Форум, 2010.448с. URL: <http://znanium.com/catalog.php?bookinfo=201030>;
3. Роббинс, Д. Отладка Windows-приложений [Электронный ресурс] / Д. Роббинс, пер. с англ. М.: ДМК Пресс, 2009. 448 с., л.URL:<http://znanium.com/catalog.php?bookinfo=407747>.

Интернет-ресурсы:

1. Технология программирования: [Электронный ресурс]. URL: [http://glossary.ru/cgi-bin/gl_sch2.cgi?RSI\)turujoo!vwujwgssowuigto9](http://glossary.ru/cgi-bin/gl_sch2.cgi?RSI)turujoo!vwujwgssowuigto9)
2. Технология программирования: [Электронный ресурс]. URL:<http://2programmer.ru/l1?start=3>
3. Delphi и базы данных: [Электронный ресурс]. URL: <http://snkey.net/books/delphi/ch4-2.html> Базы данных: [Электронный ресурс]. URL: <http://www.sdteam.com/5>